Front-End Design of String Conundrum: A Visual, Misconception-Targeted, and isiXhosa-Localised Tutor for Novice Programmers Learning Python Strings

Hlomla Kuhlane
KHLHLO002@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Abstract

Novice programmers struggle to develop accurate mental models of Python strings, yet existing tools provide limited support for string-specific misconceptions. This paper presents String Conundrum, a web-based tutoring system built through a misconception-driven design approach. A catalogue of string misconceptions, derived from literature, curriculum, and tutoring experience, was used to derive requirements and guide the visualiser, feedback, and practice mechanisms. The system provides interpreter-verified visualisation, misconception-linked feedback, and isiXhosa pedagogical localisation that protects executable code from translation. Evaluation combined correctness testing, expert review, peer developer evaluation, user acceptance testing, and three target-user evaluation cycles (n=11, 11, 13). Mean SUS scores ranged from 79.2 to 82.5, while applied knowledge results showed successful immediate application of several Python string concepts. The study does not establish long-term learning gains.

Keywords

computing education, novice programmers, Python strings, misconceptions, interactive learning environment, interface design, usability testing, localisation

ACM Reference Format:
Hlomla Kuhlane. 2026. Front-End Design of String Conundrum: A Visual, Misconception-Targeted, and isiXhosa-Localised Tutor for Novice Programmers Learning Python Strings. In *Proceedings of Computer Science Honours Project (CSC4002W '26).* ACM, New York, NY, USA, 47 pages.

1 Introduction

Learning to program remains difficult for novice students despite decades of research and educational tooling. A systematic review of 161 courses across 51 institutions reported a mean pass rate of 67.7%, with little change over time [31]. Within this broader difficulty, strings are a comparatively underexamined concept and occur in examination tasks across institutions [16]. At the University of Cape Town (UCT), declining student performance in practical assignments and tests provided the local motivation for this project.

Difficulty with strings extends beyond syntax recall. Novice programmers can form inaccurate mental models of how strings are represented and manipulated, leading to errors in indexing, slicing, comparison, and immutability [12, 14]. Learners may treat strings as character arrays or as objects with methods, producing different expectations about how string operations behave [12]. Effective instructional support should therefore make string behaviour explicit and connect incorrect answers to the misunderstanding underlying them.

Existing tools address parts of this problem separately. Visualisers expose program state while leaving learners to interpret it independently [8]. Misconception-targeted tutors address scope, mutation, and aliasing rather than strings [15], while static-analysis and localisation approaches do not provide misconception-linked instructional support for Python strings [3, 6]. Among the tools reviewed for this project, none combines a catalogue of string-specific misconceptions with visualisation, feedback, practice, and pedagogical localisation. Section 2.5 examines this design gap in more detail.

String Conundrum addresses this gap as a software design and development project. Evidence from literature, curriculum, and tutoring experience is treated as design input rather than as the subject of an investigation into the causes of string difficulty. The misconception catalogue provides a traceable basis for how string behaviour is visualised, how incorrect answers are explained, and how practice is delivered. isiXhosa localisation is treated as part of instructional delivery while executable code is protected from translation.

1.1 Aims

Five aims guide the project. Each is linked to one or more of the nineteen functional and thirteen non-functional requirements listed with acceptance conditions in Tables B.1 and B.2, and is revisited in Section 6. Sections 2.1 to 2.5 establish the instructional needs from which these aims follow.

Aim 1. Deliver eight string-topic modules through a data-driven content architecture, so a further module requires no rendering change (FR-1, NFR-8).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CSC4002W '26, University of Cape Town, South Africa
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Aim 2. Provide an interactive visualiser and standalone Code Space, both reachable without an active module or account, resolving expressions consistently with the Python interpreter (FR-2 to FR-5).

Aim 3. Provide graded practice across four question types, with feedback naming a tagged misconception where one applies, independent of reading language (FR-6 to FR-8).

Aim 4. Deliver lesson content, prompts, hints, and feedback in isiXhosa through a pipeline where a further language needs only a locale bundle, translation falls back to English, and code is never altered (FR-8, FR-10, NFR-8, NFR-10).

Aim 5. Provide supplementary practice and progress visibility to signed-in learners, with learner-controlled leaderboard participation and display identity, and lecturer oversight that renders server-supplied cohort and per-student figures without treating unrecorded values as zero (FR-12, FR-17 to FR-19, NFR-13).

The project is evaluated using correctness testing, usability testing, expert interface review, peer developer evaluation, and user acceptance testing. The front-end contribution reported here covers module delivery, the visualiser, Code Space, practice and examination, the 276-item examination bank, localisation, Challenges, the leaderboard, and the lecturer dashboard. The lecturer dashboard also includes the back-end operations for lecturer authorisation, server-supplied analytics, and persistence of module and exercise authoring changes that form part of this contribution. Other back-end grading, execution, and persistence are documented separately [18].

The main contribution is a software design approach in which an explicit misconception catalogue provides traceability from requirements to instructional interface decisions, together with a localisation pipeline that preserves executable code while extending pedagogical content to isiXhosa.

2 Background and Related Work

The literature is organised by the role it plays in the design. Large-scale datasets establish the frequency of programming errors, task-based studies establish mental models at smaller scale, theoretical accounts explain persistence without prescribing intervention, and localisation studies establish language as a barrier outside any specific topic. Section 2.5 reviews existing tools and what none of them provides.

2.1 Strings as a Universal and Underexamined CS1 Concept

Strings are an unavoidable but comparatively underexamined concept within introductory programming. A multi-institution analysis found that strings appeared in every CS1 examination studied, with indexing and slicing assessed at all three institutions examined, and that the average examination question required approximately eleven concept components simultaneously [16]. Strings are therefore rarely assessed in isolation, and string-specific difficulties cannot readily be diagnosed from conventional assessment alone.

2.2 Empirical Evidence of String Difficulty

Several independent studies provide converging evidence of difficulty. An analysis of more than 37 million compilation events found mismatched quotation marks among the most frequent error categories, with incorrect string equality comparisons affecting 29,627 students at a median fix time exceeding 1,000 seconds [2], though as compilation logs it captures only errors producing a compiler message, not syntactically valid but conceptually wrong code. Only 49% of 338 students completed a string-manipulation task after a semester of instruction, and just 6% used string methods taught one week earlier [12], the most directly relevant source since it evaluates strings specifically rather than as one error category among many. String-formatting errors accounted for every runtime error in a first Java assignment [11], a single-institution case study, and in Python, missing string delimiters produce misleading name errors that around a quarter of affected students then respond to with incorrect advice [14]. Collectively, spanning Java, Python, and C, these studies provide stronger evidence of persistent difficulty than any one alone, while illustrating that none isolates its conceptual cause.

2.3 Why Strings Resist Instruction

The notional machine framework holds that novices struggle when their mental model of execution diverges from actual program behaviour [25], a theoretical lens rather than a string-specific finding. Evidence for divergent models exists directly: 53% of participants in the standalone task study treated strings as character arrays while 32% treated them as objects with methods [12], competing models that lead to different expectations about how operations behave. A cognitive-load perspective adds that slicing requires coordinating the original string, index positions, and the resulting substring simultaneously [30], and string concepts may be threshold concepts, where a complete understanding transforms a learner's grasp of later material while an incomplete one obstructs it [4], though this account rests on a small qualitative study. Together this literature offers explanations for why string difficulty resists instruction, though it does not prescribe a specific intervention. A broader literature review has catalogued programming misconceptions generally [22], describing them without attaching each to an interface consequence; the catalogue in Section 3.3 narrows this to strings, adds curriculum and local tutoring sources, and requires every entry to produce an identifiable design decision.

2.4 Language as a Compounding Barrier

Programming instruction is predominantly delivered in English to learners who are often not native speakers. A survey of 840 learners across 86 countries found barriers in reading instructional materials, interpreting technical communication, and learning English and programming simultaneously [9], though its self-reported, volunteer-recruited data may overrepresent motivated learners. In a stronger causal design, novices using Scratch localised to their home language learned new concepts faster than peers using the English interface [6], albeit in an informal block-based environment with younger learners rather than formal university instruction, and English-medium instruction in

African primary education has been associated with poorer performance [13], a context distinct from tertiary computer science. Together these sources establish language as a measurable barrier without prescribing how localisation should be implemented for a specific topic in university-level programming. A published framework identifies twelve localisable aspects of a programming language and observes that tools commonly localise keywords while leaving the rest untouched [27]; the instructional content surrounding the language, which this project localises, falls outside that framework's scope entirely.

The UCT Language Policy, adopted by Council in December 2024, recognises isiXhosa as an official university language and commits to resourcing it for teaching and assessment [29], aligning this project's localisation with the university's own stated objectives.

2.5 What Existing Tools Do Not Provide

Visualisation shows moderate to large learning effects, though benefits accrue more to active engagement than passive observation [20, 23]. Online Python Tutor, used by more than 200,000 people [8], exemplifies the approach but visualises state without misconception-specific guidance: a learner can observe a string method's new value but must independently connect this to immutability. The SMoL Tutor provides misconception-targeted refutation but its eleven misconceptions concern scope, mutation, and aliasing rather than strings [15]. Static analysis detects some string-comparison errors without explaining why they are wrong [3], and a review of seven coding platforms found none addressed string difficulties specifically [1]. Localised Scratch translates interface and vocabulary rather than conceptual instruction [6], and Hedy, despite fifty-plus localised languages, localises the language a learner writes rather than the lesson content they read [10]. Two department projects address adjacent ground: IsiPython localises the language and its error messages [19], and ShowAlgo animates pseudocode execution rather than string state [17]; neither couples localisation to misconception-linked instruction.

Selected from the wider survey of program visualisation systems [26] and from the literature above, the tools in Table 1 span the four approaches relevant to this problem. None of the reviewed systems was identified as combining more than two of these capabilities, and none is organised around a catalogue of string-specific misconceptions.

The reviewed tools therefore provide limited support for the instructional needs identified in Sections 2.1 to 2.4, particularly string-specific visualisation, misconception-oriented feedback, and isiXhosa localisation within a learning environment for novice Python programmers. This constitutes a design gap rather than a feature gap. Among the tools reviewed, none is organised around a catalogue of string-specific misconceptions that directly shapes its interface, feedback, and practice mechanisms, and none treats localisation as pedagogical delivery rather than as interface translation alone. Pedagogical delivery translates lesson content, prompts, hints, and feedback while protecting executable code, a distinction developed further in Section 3.4.5.

Table 1: Capabilities of existing tools against the instructional needs identified in Sections 2.1 to 2.4

Tool	Step by step vis.	Misconception feedback	String specific	Localisation scope
Online Python Tutor [8]	Yes	No	No	None
SMoL Tutor [15]	No	Yes	No	None
Static analysis tools [3]	No	No	Partial	None
Localised Scratch [6]	No	No	No	Keywords only
Hedy [10]	No	No	No	Language, not content
IsiPython [19]	No	No	No	Language, not content
String Conundrum (design target)	Yes	Yes	Yes	Content and feedback

The five aims stated in Section 1.1 follow directly from these needs, with Aim 2 answering the absence of string-specific visualisation, Aim 3 the absence of misconception-oriented feedback, and Aim 4 the treatment of localisation as interface translation alone.

2.6 Ethical, Professional and Intellectual Property Considerations

This project's user testing was approved by the UCT Faculty of Science Research Ethics Committee (SCI/03233/2026) with gatekeeper permission from the Department of Student Affairs. Participation was voluntary, uncompensated, and based on informed consent (Section 5); recruitment data was collected separately from study responses, which were recorded against a participant code with no recruitment data retained alongside them, so analysis proceeded from anonymised data. Intellectual property vests in UCT under its Intellectual Property Policy [28], and the source code is released under an open-source licence.

3 Requirements and Design

This section describes how the requirements for String Conundrum were established, how they constrained the system design, and why the principal design decisions were selected over alternatives. The discussion focuses on the front-end contribution, comprising module delivery, the string visualiser, Code Space, practice and examination interfaces, localisation, Challenges, the leaderboard, and the lecturer dashboard. Authentication, sandboxed execution, grading, misconception matching, persistence, and server-side aggregation are part of the wider back-end architecture and are documented separately [18]. The lecturer dashboard nevertheless includes back-end functionality implemented as part of this contribution, specifically the role-restricted operations, server-supplied lecturer analytics, and persistence of module and exercise authoring changes.

3.1 Requirements Elicitation

Requirements were developed from five complementary sources. Literature on string-learning difficulties informed the misconception catalogue. Consultation with Mr Aslam Safla, the project proposer and a first-year Computer Science educator at UCT, established local teaching needs. The CSC1015F syllabus and past examination papers (2019–2025) set module scope and progression. An interface agreement with the back-end developer fixed component responsibilities and boundary data. Usability testing exposed interaction requirements only during use, producing FR-13 to FR-16 for narration and error presentation and FR-17 to FR-19 with NFR-13 for lecturer oversight. Requirements were consequently treated as an evolving specification within a stable architectural boundary.

3.2 Functional and Non-Functional Requirements

Nineteen functional requirements, twelve specified initially plus FR-13 to FR-16 from Cycle 1 and FR-17 to FR-19 for lecturer oversight, are listed with acceptance conditions in Table B.1. The non-functional requirements in Table B.2 constrain architecture rather than merely describing desirable qualities. NFR-8 requires new modules and languages without modifying existing rendering components; NFR-9 keeps grading and execution server-side; NFR-10 requires field-level English fallback; NFR-12 requires visualiser state to be communicated without relying on colour alone; and NFR-13 extends server authority to lecturer-facing marks and statistics. The design decisions that follow refer to the requirements that materially constrain them, establishing traceability from requirement to design, implementation, and acceptance testing.

3.3 Misconception Catalogue as a Design Input

Twenty-four misconceptions were catalogued across string fundamentals, indexing, slicing, immutability, methods, and comparison, as shown in Table A.1, drawing on literature, the CSC1015F curriculum, supervisor consultation, and locally observed tutoring experience. Rather than remaining a passive list, the catalogue was used to derive identifiable interface and feedback decisions from specific learner difficulties. The misconception that a slice includes its stop bound led to explicit start and stop boundaries in the visualiser with the stop position visibly excluded. Negative-index confusion led to simultaneous positive and negative index rows rather than a textual note, while confusion between counting and locating substrings informed feedback that names the relevant misconception rather than only reporting an incorrect answer.

The same vocabulary supports procedural practice. Each Practice Engine template carries a misconception identifier, but a general template tag can be overridden by a finer diagnostic check. For a generated attempt, the system recomputes the result associated with a candidate misconception and compares it with the learner's response, permitting a more specific diagnosis while preserving one vocabulary across authored and procedurally generated exercises. The catalogue serves as the primary traceability mechanism from educational evidence to interface behaviour, diagnostic feedback, and practice generation.

3.4 System Design

3.4.1 Methodology. Development followed a hybrid methodology [24]. Initial requirements, architecture, and the front-end and back-end interface contract were established up front, while implementation, testing, and learner-facing requirements evolved iteratively. A purely sequential process was rejected because learner reactions could not be determined during specification alone, demonstrated when Cycle 1 produced FR-13 to FR-16 and prompted procedural practice generation. A wholly iterative process was also unsuitable because the cross-developer interface contract needed to remain stable enough for parallel development. The hybrid approach therefore stabilised the shared boundary while allowing the front-end artefacts and requirements to evolve in response to evaluation.

3.4.2 Architecture. Presentation is separated from processing across a same-origin network boundary, shown in Figure 1 with endpoint-level detail in Figure C.1. The front end owns presentation and interaction and never computes a mark; the back end owns authentication, execution, and grading and is the only component that reads answer keys and misconception mappings.

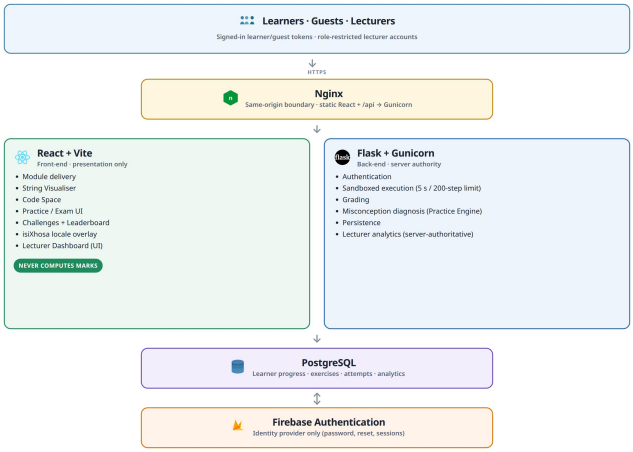


Figure 1: System architecture showing the separation of concerns between front-end and back-end components, with the same-origin boundary enforced by Nginx.

Server-side grading and execution were selected over browser-side evaluation because a browser-delivered answer key would be inspectable by a learner and centralised execution allowed the required resource limits to be enforced consistently. The execution service applies a five-second limit and a 200-step tracer cap under NFR-9, and submissions are evaluated as programs rather than compared against fixed answer strings, permitting multiple valid solutions credit. The same authority boundary applies to lecturer-facing analytics. The browser derives no marks, completion percentages, misconception frequencies, or pass/fail decisions from raw learner records; the back end supplies these figures instead, preventing the presentation layer from becoming a second grading implementation (NFR-13).

3.4.3 Technologies. The main technology choices for String Conundrum were made jointly by the two developers based on the

requirements of the shared system. React was selected for the front end because the interface is organised around reusable stateful components shared across the visualiser, module stages, practice interfaces, Code Space, and lecturer views. Angular was considered but provided a broader framework than was required for the application structure. Four React contexts carry the limited cross-screen state, so an additional state-management library was not required. Vite was selected as the development and build environment because it supported rapid iteration during front-end development.

Firebase Authentication was selected rather than implementing authentication directly because it provides established password, verification, and session-management mechanisms while keeping this security responsibility outside the educational application. Flask was selected for the server because the project required a lightweight Python API for execution, grading, and data exchange, while Django included additional framework features that were not required. PostgreSQL was selected over a document-oriented alternative such as Firestore because learner, module, exercise, and attempt records have clear relationships that are naturally represented and constrained through a relational schema.

Implementation responsibilities were then divided between the two contributions. The front end consumes authentication, execution, grading, and persistence services through the agreed interface contract, while their server-side implementation is documented separately [18]. NLLB-200 [21] was selected for generation of the isiXhosa instructional content because the project required a repeatable offline translation process rather than a runtime dependency on a translation service. The alternatives and constraints behind this localisation decision are discussed in Section 3.4.5.

3.4.4 Content and Exercise Architecture. Each module follows a five-stage sequence (Learn, Examples, Visualise, Practice, and Exam) that moves learners from explanation through worked examples to practice and assessment. A single per-module hard-coded interface was rejected because adding a module would require changing existing rendering logic, conflicting with NFR-8, so content is represented as data and rendered through shared components. Examination draws on a fixed bank of 276 exercises sharing a schema and submission path with practice. Practice originally used a fixed set of hand-authored questions, but Cycle 1 exposed two problems. The bounded pool limited repeated practice on unmastered concepts, and each question needed its own hand-written misconception mapping. The Practice Engine was therefore redesigned to generate a new question from a template and a random word on each attempt, retaining the fixed bank's misconception identifiers while the diagnostic recomputation of Section 3.3 determines whether a more precise misconception applies (Figure C.3).

Progression between modules requires 80 percent in the preceding examination, since later topics depend on earlier concepts. A lower or optional threshold was rejected as weakening the mastery signal, and an unconditional gate was rejected because one difficult question could discourage persistence, so free navigation is available as a setting while failed attempts direct the learner to relevant practice.

3.4.5 Localisation Architecture. Localisation is part of the content architecture rather than an interface translation layer. Locale bundles apply as field-level overlays over an English base, satisfying NFR-10 since a missing translation falls back to English at the individual field. Complete parallel content files were rejected as making translated content dependent on completeness and letting the structures diverge, so content is translated ahead of time using NLLB-200 and committed to the repository (Figure E.1). Runtime translation was rejected as placing a translation service on the interaction path and preventing text from being reviewed as stable before delivery, and runtime large-language-model translation was additionally rejected because non-deterministic output would make reviewed text unstable across generations.

Executable Python is excluded from translation. A line parsing as valid Python is held back rather than identified with a regular expression, and embedded code fragments within prose are masked before translation, with a verification pass rejecting any translation that alters a protected fragment and falling back to English for that segment, necessary since many of the 276 prompts mix raw Python into prose with no code fencing. Code is instead handled by a deterministic keyword transpiler covering 58 pairs, replacing longest-first to avoid partial-word collisions and tokenising comments and string literals so a keyword inside a learner's string is untouched. The learner-facing keyword reference is generated from the same mapping, satisfying FR-10.

3.4.6 User Interface Design. A persistent sidebar exposes the principal areas, with learner-restricted areas shown as locked rather than hidden so guests can see what an account unlocks. The lecturer dashboard is the deliberate exception and is hidden from learner navigation because it contains role-restricted administrative functions. The visualiser supports Slicing, Index Access, For Loop, While Loop, If Statement, and String Method modes, allowing string behaviour to be explored both directly and within common control-flow contexts. For indexing and slicing, positive and negative indices are displayed simultaneously and inclusive and exclusive slice boundaries are labelled explicitly, directly implementing the misconception-driven decisions of Section 3.3. Under NFR-12, visual information is communicated through labelled elements and structure as well as colour and is exposed to assistive technology.

Code Space is an independent execution surface under FR-5 rather than a second graded interface. It returns output and errors without assigning a mark, distinguishing exploratory execution from Practice and Exam. Its editor includes a transpile control for isiXhosa keyword form with a visible confirmation banner, while the visualiser pane states the scope of what it can trace so that unsupported code is not mistaken for an empty result.

Spoken narration, introduced through FR-13 to FR-15 after Cycle 1, advances on speech completion rather than a fixed timer and reports explicitly when narration is unavailable in the selected language. Challenges, consisting of 18 problems with difficulty bands, hints, and test cases, and the leaderboard provide supplementary practice and progress visibility. Leaderboard participation and display identity are controlled separately by the learner so that appearing in public rankings is optional rather than the default.

3.4.7 Design Patterns and the Lecturer Dashboard. The open-closed principle governs extensibility [7]. Content blocks and exercise types are selected by declared type and share a common schema, so a new module, exercise type, block type, or language is introduced through new data and handlers without modifying existing rendering components, establishing a traceable chain from evidence to requirement to design to implementation to verification.

The lecturer dashboard is a role-restricted subsystem, available only to the authenticated lecturer role, with six functional areas (Overview, Students, Marks, Analytics, Modules, Content) that share the application's request layer. It separates read-oriented oversight from write-oriented authoring. Marks and Analytics render back-end-reported figures directly without client-side aggregation, preserving the server-authority boundary of NFR-13, and distinguish an unrecorded value from a recorded zero (FR-19) so absence of evidence is never presented as a result. The content-management interface lets lecturers create, edit, reorder, publish, and hide modules and exercises (FR-18) through the same schema and validation used by the learner-facing paths. Lesson prose and worked examples remain outside this authoring scope, since supporting arbitrary authoring there would risk inconsistency between the English and isiXhosa content the overlay system depends on.

3.4.8 A Recurring Design Principle. A further principle recurs across otherwise independent decisions in this design: an interface must never present the absence of a result as if it were one. The visualiser reports a failed evaluation as an error rather than an empty selection a learner could mistake for a legitimate answer (Section 4.3); narration explains rather than silently disables when a language is unsupported (FR-15); progress marking is issued without awaiting confirmation while grading is always awaited, so a rejected submission is reported as a failure rather than silently counted as wrong (Section 4.7); and the lecturer dashboard distinguishes an unrecorded value from a recorded zero (FR-19) as described above. None of these decisions was derived from a single requirement; the principle emerged from treating an ambiguous interface state as itself a source of misconception, in the same way the misconception catalogue treats an ambiguous slice boundary.

4 Implementation

This section describes the implementation that realises the design decisions of Section 3.4.

4.1 Development Overview

Development proceeded through working increments after the front-end and back-end interface contract fixed the data exchanged across the system boundary. These increments covered the application shell, module delivery, the visualiser, Code Space, practice and examination, isiXhosa delivery, Challenges, the leaderboard, and the lecturer dashboard. Cycle 1 target-user evaluation ran against the system at that state, so its findings shaped the implementations below rather than arriving after they were settled (Section 4.9). Additional interface views are provided in Supplementary Figures L.1–L.8.

4.2 Module Delivery

A module is defined by a content file registered against a module identifier. Each content block is dispatched to a renderer according to its declared type, allowing new modules to be added without changing existing rendering logic. Progress is recorded per stage rather than per exercise. This replaced an earlier rule that treated a module as complete only when every exercise had been answered correctly. The stage record drives the progress indicator and is used when applying the 80% progression rule.

4.3 String Visualiser

The visualiser exists as a standalone screen and an embedded form at the Visualise stage, both built on one rendering component and one slicing and indexing engine (Figure 2). The visualiser supports six modes covering Slicing, Index Access, For Loop, While Loop, If Statement, and String Method operations. These modes allow learners to inspect direct string operations as well as common control-flow contexts in which strings are accessed and manipulated. For slicing and indexing, the engine resolves an expression to selected positions in the browser and never calls the execution service or assigns a mark (Figure C.5). Correctness is treated with particular weight, since a visualiser that resolves a slice incorrectly teaches precisely the misconception it exists to correct; it is verified against the Python interpreter rather than the module's own reading of the language rules, as Section 4.8 sets out, and failed evaluations report the failure rather than an empty selection a learner could read as a legitimate answer.

Spoken narration uses the browser's own speech synthesis, requiring no network dependency, and advances on utterance completion rather than a fixed interval (NFR-11). Narration is deliberately unavailable in isiXhosa, since no major browser ships an isiXhosa voice and a substituted South African English voice would mispronounce the lesson while appearing to have worked, so activating it opens a dialog stating the limitation (FR-15) rather than silently disabling the control. The visualiser now plays automatically on entering the Visualise stage, removing a manual trigger Cycle 1 participants had to discover unaided (Section 4.9).

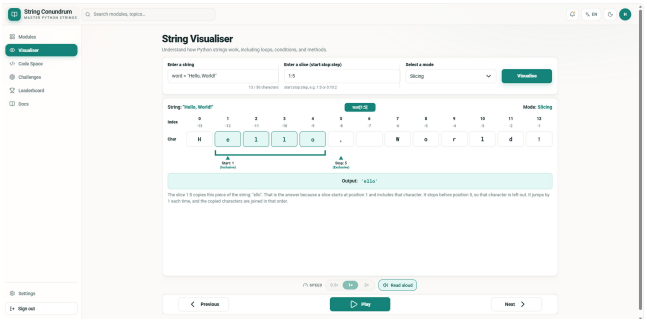


Figure 2: The string visualiser showing positive and negative index schemes, slice bounds (start inclusive, stop exclusive), and the output pane with explanation.

4.4 Code Space

Code Space provides an editor and execution surface independent of the module sequence, reachable without an account via the guest mechanism of Section 3.4.2 (Figure C.2). Submitted code is sandboxed with a five-second limit and a 200-step tracer cap [18], and Code Space returns output only, performing no grading, unlike Practice and Exam which evaluate against a server-held answer key (NFR-9). A transpile control converts the editor's contents into isiXhosa keyword form with a confirmation banner, and a visualiser pane states the scope of what it traces rather than leaving a learner to misread an empty result as evidence that arbitrary Python cannot be visualised.

4.5 Practice and Examination

Examination uses a fixed bank of 276 exercises, while Practice generates questions procedurally across all eight modules from templates and randomly selected words. Both use the same exercise schema, submission path, and misconception identifiers. Diagnosis, pattern matching, and weak-point weighting are provided by back-end services [18] and consumed through a shared feedback function, so equivalent errors in generated and hand-authored exercises receive the same explanation.

A worked-example panel accompanies both correct and incorrect answers, showing the same character-cell highlighting as the visualiser alongside the rule that produced the correct result and a contrasting case with different bounds (Figure 3). The browser keeps a running tally of which misconception id, if any, a wrong answer was diagnosed against, sent with each request so subsequent questions weight toward a learner's demonstrated weak points.

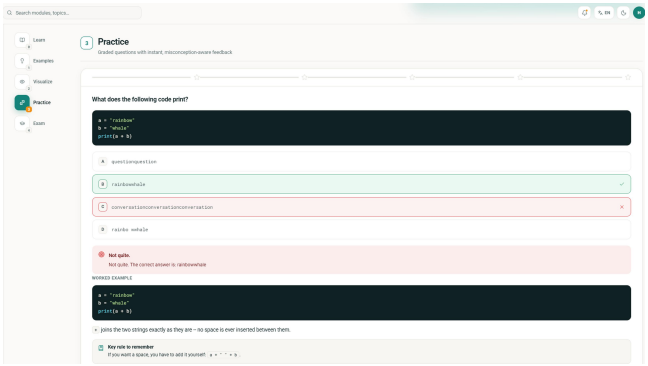


Figure 3: Example of misconception-linked feedback showing why an answer is incorrect, with a visual aid in the worked example panel.

4.6 isiXhosa Delivery

The active locale is persisted in browser storage, inferred from the browser's language preference where none exists, and applied to the document's language attribute for assistive technology. A language control is available before sign-in as well as from settings, since a learner who reads isiXhosa needs the interface in isiXhosa before holding an account. Interface strings resolve through a dotted-key lookup falling back to English and then to the key itself (NFR-10), and module content resolves through a deep merge of the isiXhosa overlay over the English base, merging arrays by position so an overlay can translate part of a section and leave the rest in English rather than failing wholesale; all eight modules include isiXhosa overlay content.

Natural-language content is translated ahead of time and committed so that no translation model runs on any request path. Code is instead handled by the deterministic keyword transpiler, whose mapping also generates the learner-facing keyword reference (FR-10, Figure 4). Both are implemented as specified in Section 3.4.5; Figure E.1 shows the offline pipeline.



Figure 4: The same program shown in Python (English keywords) and isiXhosa (transpiled keywords).

4.7 Challenges, Leaderboard, and Back-End Integration

Challenges presents 18 problems, each with a difficulty band, hint, worked solution, and test cases, evaluated through the same execution service as Code Space (Figure C.4), with a line-number gutter added after Cycle 1 (Section 4.9). Leaderboard participation uses separate settings for whether a learner appears at all and under what identity, leaving that judgement with the learner rather than defaulting to public ranking; both areas are shown locked in guest mode.

Requests pass through a single wrapper that attaches a server-verified bearer token, so the leaderboard identity originates from authenticated server data rather than from a browser-controlled field. Progress updates are sent without blocking navigation because failure to record progress does not affect the correctness of the learner's current interaction. In contrast, grading, execution, and challenge submissions are awaited because their results determine the feedback shown to the learner. A rejected request is therefore reported explicitly rather than being silently treated as an incorrect answer.

4.8 Development Testing

Coverage is not uniform. Rendering behaviour is not exercised, so a component that computed the right answer and displayed it wrongly would pass, and the browser-side weak-point tally has no automated test coverage, stated here as a known gap rather than a later discovery. The tests instead address the pure logic on which the paper's correctness claims rest. The suite uses the JavaScript runtime's own test runner and contains 133 tests across 37 suites, covering the indexing and slicing engine, the localisation merge, the mastery gate, the shared exercise helpers, the keyword transpiler in both directions, syntax highlighting, and account validation. All 133 pass, as shown in Figure D.1, and the suite is broken down by area and requirement in Table D.1.

The engine tests are not based only on expected values derived from the implementation. A reference set of 132 slice expressions and 60 index expressions was evaluated by the Python interpreter, and the test suite checks that the visualiser reproduces those results. This reduces the risk of the implementation and its tests sharing the same incorrect interpretation of Python semantics. Additional tests verify properties on which the design depends, including that a slice excludes its stop position, negative indices count from the end of the string, an out-of-range index raises an error while out-of-range slice bounds are clamped according to Python slicing semantics, a step of zero produces Python's own error, untranslated fields fall back to English, and module progression depends on examination rather than practice. Continuous integration runs the front-end suite before the application is built, alongside the separately developed back-end suite [18]. The Practice Engine's template generation, misconception diagnosis, and weakpoint weighting are tested separately on the back end.

4.9 Feedback-Related Changes

Cycle 1 target-user evaluation produced eleven changes, each recorded against the finding that prompted it (Table I.1). The most substantial was spoken narration, requested independently by two participants, once for audio learners and once on behalf of blind users (Section 4.3), followed by the replacement of the fixed practice bank with the Practice Engine (Section 4.5). Findings not yet resolved are listed in Table I.2 and discussed in Section 7.

5 Evaluation Methods

Evaluation combined four complementary forms of evidence. Target-user cycles examined learner-facing usability and demonstrated understanding during interaction. Four HCI postgraduate students provided qualitative interface review, three PyTutor developers provided an independent technical perspective, and the project supervisor conducted user acceptance testing against the functional and non-functional requirements. These methods were combined because no single method could establish implementation correctness, usability, requirement satisfaction, and learner-facing understanding at the same time. Detailed instruments, procedures, method rationale, and decision records are provided in the Supplementary Information.

5.1 Target-User Evaluation

Three target-user cycles were conducted. Cycles 1 and 2 each involved 11 second- and third-year undergraduate students and used the same core instrument. Cycle 3 involved 13 students enrolled in the first-year course CSC1016S, including three extended-programme participants, and took place immediately after a Java practical session in the same room and scheduled session. Participants interacted with two of the eight modules for approximately 30 minutes, completed the relevant examination activities, and then completed the evaluation instrument. A researcher recorded points of confusion and assisted only when a participant could not otherwise proceed.

Cycles 1 and 2 used retrospective confidence ratings, an applied knowledge check, open reflection, interface feedback, and the ten-item System Usability Scale (SUS) [5]. Cycle 3 retained SUS unchanged but revised the confidence and agreement items and added a method-mutation knowledge question. The three cycles were therefore iterative evaluation points rather than identical replications. SUS is compared descriptively across all cycles, while comparisons involving the revised Cycle 3 measures are interpreted more cautiously.

SUS scores were calculated using the standard ten-item scoring procedure and summarised descriptively using means and medians. Applied-knowledge responses were scored as fully correct or not fully correct by concept, with 95% Wilson confidence intervals used to show uncertainty around proportions. Confidence and agreement ratings were summarised descriptively because the samples were small and the Cycle 3 instrument differed from the earlier cycles. Open-response and observation data were grouped by recurring issue or feature rather than treated as numerical measures.

5.2 Expert, Peer, and Acceptance Evaluation

The four HCI reviewers examined navigation, the visualiser, Code Space, module screens, and examination screens. No numerical expert scale was used, so the findings are treated as qualitative interface evidence. The three PyTutor developers completed a six-question open-response review before Cycle 1 covering strengths, weaknesses, confusing elements, and design differences. Their responses are treated as qualitative peer feedback rather than a numerical comparison.

User acceptance testing was conducted with the project supervisor against the agreed functional and non-functional requirements. The production system was exercised directly, including requirements not reached during unprompted learner navigation. UAT therefore provides requirement-level evidence that the implemented system satisfies the agreed specification. Because the supervisor was also the requirement originator, this evidence is treated as acceptance evidence rather than an independent usability or educational-effectiveness assessment.

6 Results

This section reports the outcomes of the evaluation methods described in Section 5. Development-test evidence is reported separately in Section 4.8. Detailed participant-level results, instruments, and supplementary analyses are provided in the Supplementary Information.

6.1 Misconceptions Observed During Evaluation

Across all three cycles, seven participants produced the fabricated result "Jython" when predicting the effect of `word[0] = "J"`. The response occurred in every cycle, making in-place string mutation a recurring misconception-aligned error within the evaluated samples. This does not establish why the error occurs or whether the visualiser corrects it. Slice-stop inclusivity and indexing from one also recurred across the evaluation.

Cycle 3 additionally produced a response not represented in the original catalogue. One participant expected the invalid assignment to make no change and raise no error. This differs from expecting the string to mutate and suggests a separate feedback case.

The recurring catalogued errors and this additional response support using the catalogue as a design vocabulary while keeping it open to refinement.

6.2 Target-User Outcomes

6.2.1 Perceived Usability and Confidence. Complete SUS responses were obtained from 10 of 11 participants in Cycle 1, 10 of 11 in Cycle 2, and all 13 participants in Cycle 3. Mean SUS scores were 79.2, 82.5, and 79.8, with medians of 82.5, 88.75, and 80.0 respectively. SUS was the only measure retained unchanged across all three cycles, and the scores remained similar despite differences in participant population and evaluation context. Because the samples were small and non-equivalent, this similarity is treated descriptively rather than as evidence that usability is independent of cohort or setting. Figure 5 summarises the scores, with score distributions provided in Supplementary Figure J.6.

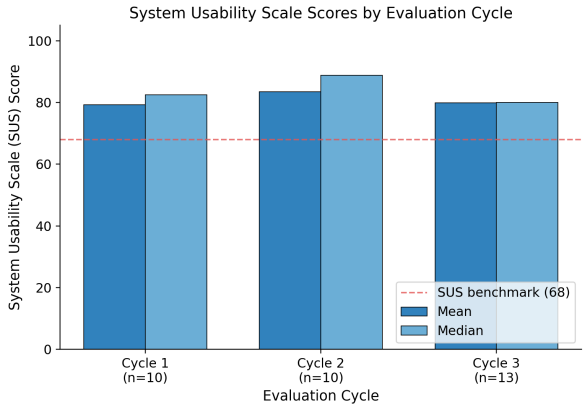


Figure 5: System Usability Scale scores across the three evaluation cycles. Bars show the mean and median SUS scores for each cycle.

Retrospective confidence increased across the concepts measured in each cycle. The largest increases in Cycles 1 and 2 concerned immutability, with gains of 1.80 and 1.63 on the five-point scale. Because confidence was measured retrospectively rather than through a matched pretest, these results describe perceived change and do not establish a causal learning gain. Full confidence profiles are provided in Supplementary Figures J.1 and J.2.

6.2.2 Applied Knowledge. Applied knowledge results showed strong performance on several string concepts. In Cycle 2, indexing, slicing, and comparison each reached 100% fully correct. In Cycle 3, indexing was correct for 12 of 13 participants, while slicing and comparison were each correct for 10 of 13. Cycle 3 also tested method mutation, with 11 of 13 participants correctly identifying that `upper()` leaves the original string unchanged. Immutability had lower fully correct results, with 5 of 11, 7 of 11, and 8 of 13 correct across the three cycles. Because participants completed only two of the eight modules, these responses should not be treated as a direct measure of the immutability module's effectiveness.

Figure 6 summarises the recurring concepts. Wilson 95% confidence intervals are provided in Supplementary Table G.4. Their width reflects the small samples and limits precise comparison between cycles.

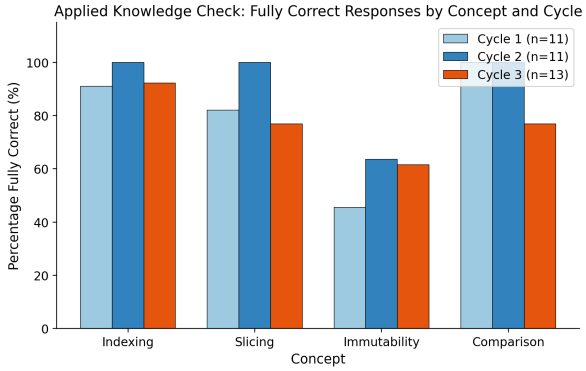


Figure 6: Percentage of fully correct responses on the applied knowledge check by concept and evaluation cycle.

6.2.3 Qualitative Findings. The visualiser, index and slice highlighting, and misconception-linked feedback were repeatedly identified positively. In Cycle 3, index and slice highlighting received the highest feature rating, followed by the visualiser and misconception-linked feedback, as shown in Figure 7.

Participants also identified recurring areas for refinement. Some isiXhosa terminology was unclear or inconsistent, some participants were unsure whether answers had been graded correctly, Cycle 3 participants requested broader topic coverage and difficulty levels, and participants in Cycles 1 and 3 reported initial confusion when using Code Space. These findings distinguish technical correctness from the clarity and trustworthiness of the learner-facing experience.

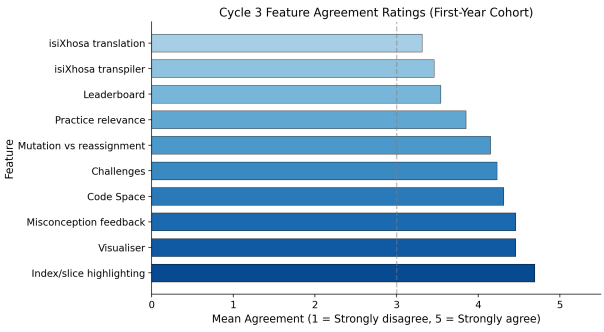


Figure 7: Mean agreement ratings for key features in Cycle 3 for the First-Year cohort. Ratings use a 1–5 scale where 5 indicates strong agreement.

Perceived isiXhosa helpfulness varied across cycles, with mean ratings of 3.00 in Cycle 1, 3.90 in Cycle 2, and 3.70 among Cycle 3 respondents. In Cycle 3, agreement that the translation made the content easier had a mean of 3.31, while the isiXhosa transpiler received a mean rating of 3.46. These results indicate perceived

value for some participants, while the variation and qualitative comments reinforce the need for independent language review. Supplementary Figures J.3–J.5 provide the detailed localisation results.

6.3 Expert, Peer, and Acceptance Outcomes

HCI reviewers generally described navigation as clear and the interface as coherent, while identifying issues involving consistency, information hierarchy, and interaction feedback. All three peer developers identified the visualiser positively. Their main concerns involved the examination bank, including repetitive questions, incorrect marking, missing correct-answer options, and limited negative-index coverage. They also identified weaknesses in Challenge error explanations and lecturer-facing weak-point information.

The supervisor accepted the exercised requirements and feature set, including all eight modules, Code Space, the visualiser, the four grading types, misconception-linked feedback, localisation, Challenges, the leaderboard, settings, and lecturer-facing functionality. One sign-up item was not recorded, and one escape-sequence rendering issue identified during UAT was corrected before submission. The full UAT record is provided in the Supplementary Information.

6.4 Results Against the Project Aims

Taken together, the results provide evidence that the implemented system addressed the five design aims within the tested scope. Development testing and UAT provide evidence for implementation correctness and requirement satisfaction. SUS and qualitative observations show generally positive learner-facing usability, while the applied knowledge results provide evidence that participants could successfully apply several Python string concepts during the evaluation sessions.

These findings support immediate conceptual understanding within the evaluated sessions, but do not establish durable learning gains because the cohorts were small and non-equivalent, participants experienced only two of the eight modules, confidence was measured retrospectively, and the Cycle 3 instrument differed from the earlier versions. Supplementary Table K.5 maps each aim to its supporting evidence and interpretation boundary.

7 Discussion

The evaluation provides a positive but bounded picture of String Conundrum as an educational software system. The results support implementation correctness, requirement satisfaction, positive perceived usability, and immediate application of several Python string concepts during the target-user sessions. Recurring learner errors also corresponded to entries in the misconception catalogue, supporting its use as a basis for instructional design. The short evaluation sessions, small non-equivalent samples, and limited module exposure restrict claims about longer-term learning, but do not diminish the evidence of demonstrated understanding within the evaluated sessions.

7.1 Misconception-Driven and Iterative Design

The recurring "Jython" response, slice-stop inclusivity, and indexing-from-one errors show that several catalogue entries also appeared during target-user evaluation. This supports the catalogue's usefulness as a design vocabulary for deciding what the interface should expose and what feedback should explain. It should not, however, be treated as a causal model of novice behaviour. The additional uncatalogued Cycle 3 response reinforces the need for the catalogue to remain extensible rather than fixed.

Evaluation also changed the artefact. Cycle 1 led to narration, automatic visualiser playback, clearer examination output, shuffled answer options, Challenge line numbers, and replacement of the fixed practice bank with the Practice Engine. Cycle 2 prompted further refinements involving string-length visibility, keyword explanations, contrast, syntax-error handling, slicing explanations, and practice variety. This provides traceability from observed problems to implemented design responses and strengthens the software-engineering contribution. It also means that the three cycles evaluated different versions of the artefact and should not be treated as identical replications.

7.2 Usability and Demonstrated Understanding

Mean SUS scores remained similar across the three cycles, and qualitative feedback repeatedly identified the visualiser, index and slice highlighting, and misconception-linked feedback positively. These findings support generally positive perceived usability within the evaluated sessions. Applied knowledge results also showed successful immediate application of several string concepts, particularly indexing, slicing, comparison, and method behaviour.

Performance was not identical across all concepts, showing that usability and conceptual understanding provide complementary evidence rather than one acting as a proxy for the other. This distinction is important because the project was intended both to provide a usable learning environment and to support learners in reasoning about Python string behaviour.

Cycle 3 differed from the earlier cycles in participant population and evaluation context, followed a CSC1016S Java practical, and used revised confidence and knowledge measures. Differences between cycles are therefore interpreted as differences observed under changing evaluation conditions rather than evidence that the system became more or less effective over time.

Peer evaluation also exposed weaknesses in the fixed examination bank, including repetition, incorrect marking, missing correct-answer options, and limited negative-index coverage. These issues do not affect the procedural Practice Engine in the same way, but they show that the fixed examination bank still requires systematic content validation. The findings also support the decision to replace fixed-bank practice with procedural generation while retaining further examination-bank review as future work.

7.3 Localisation and Evaluation Boundaries

Development testing supports the technical localisation pipeline, including field-level English fallback, protection of executable code, and deterministic keyword transpilation. User feedback also

showed perceived value in the isiXhosa support, although some terminology was reported as unclear or inconsistent. Technical correctness of the localisation mechanism and linguistic quality of the translated content are therefore separate concerns. Independent isiXhosa language review remains necessary to assess the latter more fully.

The evaluation also has limitations that constrain interpretation. The samples were small and differed across cycles, each participant experienced only two of the eight modules in a single session, and confidence was retrospective rather than measured through a matched pretest. Expert review was qualitative, UAT was conducted by the requirements originator, and complete automated rendering coverage remains outstanding. These limitations restrict claims about generalisability and longer-term learning, but the target-user results still provide evidence of immediate demonstrated understanding within the evaluated sessions.

The main contribution is a tested educational software design approach connecting identified string misconceptions to visualisation, feedback, practice generation, and localisation. The evaluation provides encouraging evidence of successful use and demonstrated understanding within the tested sessions.

8 Conclusions and Future Work

String Conundrum was developed as an educational software system to help novice programmers understand Python string concepts. Its misconception-driven design connects learner difficulties to visualisation, explanations, practice, feedback, and localisation. The completed system combines eight learning modules, an interpreter-verified visualiser, Code Space, misconception-linked feedback, procedural practice, isiXhosa instructional localisation, Challenges, a learner-controlled leaderboard, and lecturer-facing functionality.

The evaluation provides positive evidence that the five project aims were addressed within the tested scope. Development testing and UAT supported implementation correctness and requirement satisfaction, while target-user evaluation showed positive usability and immediate application of several string concepts. Participants performed strongly on indexing, slicing, comparison, and method behaviour, and the visualiser, highlighting, and misconception-linked feedback were repeatedly received positively.

Overall, String Conundrum demonstrates how misconception-targeted instruction, visualisation, practice, feedback, and localisation can be combined into a coherent learning environment for Python strings. The study does not establish long-term learning gains because participants used only part of the system during short sessions, but it provides encouraging evidence of immediate understanding. Future work should evaluate more modules over longer periods, use matched pre- and post-tests, review the isiXhosa content independently, and continue refining the examination bank.

9 Generative AI Usage

ChatGPT and Google Gemini were used as supplementary writing and learning tools during the preparation of this paper. They were used to review and improve my own writing across the Introduction, Background, Requirements and Design, Implementation, Evaluation, Results, Discussion, Conclusions, and supplementary material, particularly for grammar, clarity, structure, organisation, and identifying unclear or repetitive wording. They were also used to support my understanding of LaTeX and the formatting and organisation of tables, figures, appendices, and evaluation data.

AI-generated suggestions were not accepted automatically. I checked, edited, or rejected suggestions against the project implementation, collected data, and source material. I remained responsible for the project design, implementation, evaluation procedure, data accuracy, interpretation, arguments, and final content.

References

[1] O. Alasmari, J. Singer, and M. Bikanga Ada. Do Current Online Coding Tutorial Systems Address Novice Programmer Difficulties? In *Proc. ICETC 2023*. ACM, New York, 242–248. https://doi.org/10.1145/3629296.3629333

[2] A. Altadmri and N. C. C. Brown. 37 Million Compilations: Investigating Novice Programming Mistakes in Large Scale Student Data. In *Proc. SIGCSE 2015*. ACM, New York, 522–527. https://doi.org/10.1145/2676723.2677258

[3] T. Blok and A. Fehnker. Automated Program Analysis for Novice Programmers. In *Proc. HEAd 2017*. Universitat Politècnica de València, 1–8. https://doi.org/10.4995/HEAD17.2017.5533

[4] J. Boustedt, A. Eckerdal, R. McCartney, J. E. Moström, M. Ratcliffe, K. Sanders, and C. Zander. Threshold Concepts in Computer Science: Do They Exist and Are They Useful? In *Proc. SIGCSE 2007*. ACM, New York, 504–508. https://doi.org/10.1145/1227310.1227482

[5] J. Brooke. 1996. SUS: A Quick and Dirty Usability Scale. In *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland (Eds.). Taylor & Francis, London, UK, 189–194.

[6] S. Dasgupta and B. M. Hill. Learning to Code in Localized Programming Languages. In *Proc. LS 2017*. ACM, New York, 33–39. https://doi.org/10.1145/3051457.3051464

[7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1995.

[8] P. J. Guo. Online Python Tutor: Embeddable Web-Based Program Visualization for CS Education. In *Proc. SIGCSE 2013*. ACM, New York, 579–584. https://doi.org/10.1145/2445196.2445368

[9] P. J. Guo. Non-Native English Speakers Learning Computer Programming: Barriers, Desires, and Design Opportunities. In *Proc. CHI 2018*. ACM, New York, 1–14. https://doi.org/10.1145/3173574.3173970

[10] F. Hermans. Hedy: A Gradual Language for Programming Education. In *Proc. ICER 2020*. ACM, New York, 259–270. https://doi.org/10.1145/3372782.3406262

[11] M. C. Hughes, M. C. Jadud, and M. M. T. Rodrigo. String Formatting Considered Harmful for Novice Programmers. *Computer Science Education* 20, 3 (2010), 201–228. https://doi.org/10.1080/08993408.2010.507335

[12] C. Izu and A. Weerasinghe. Assessing CS1 Design Skills with a String Manipulation Task. In *Proc. ITiCSE 2020*. ACM, New York, 432–438. https://doi.org/10.1145/3341525.3387382

[13] A. N. Kioko, R. W. Ndung'u, M. C. Njoroge, and J. Mutiga. Mother Tongue and Education in Africa: Publicising the Reality. *Multilingual Education* 4, 1 (2014), Article 18. https://doi.org/10.1186/s13616-014-0018-x

[14] T. Kohn. The Error Behind the Message: Finding the Cause of Error Messages in Python. In *Proc. SIGCSE 2019*. ACM, New York, 524–530. https://doi.org/10.1145/3287324.3287381

[15] K.-C. Lu and S. Krishnamurthi. Identifying and Correcting Programming Language Behavior Misconceptions. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), Article 106, 28 pages. https://doi.org/10.1145/3649823

[16] A. Luxton-Reilly and A. Petersen. The Compound Nature of Novice Programming Assessments. In *Proc. ACE 2017*. ACM, New York, 26–35. https://doi.org/10.1145/3013499.3013500

[17] G. A. Maselle. Implementing Effective Animated Visualisations of Pseudocode. Honours Project Paper, Department of Computer Science, University of Cape Town, Cape Town, South Africa, 2024.

[18] L. Matomela. String Conundrum: An interactive System for Teaching and Visualising Python String Concepts. Honours Project Paper, Department of Computer Science, University of Cape Town, Cape Town, South Africa, 2026.

[19] L. Mngqibisa. IsiPython Backend: Enabling Python Programming in isiXhosa Through Localisation and LLM-Augmented Error Translation. Honours Project Paper, Department of Computer Science, University of Cape Town, Cape Town,

South Africa, 2025.

[20] A. Moreno, N. Myller, E. Sutinen, and M. Ben-Ari. Visualizing Programs with Jeliot 3. In *Proc. AVI 2004*. ACM, New York, 373–376. https://doi.org/10.1145/989863.989928

[21] NLLB Team, M. R. Costa-jussà, J. Cross, O. Çelebi, M. Elbayad, K. Heafield, K. Heffernan, E. Kalbassi, J. Lam, D. Licht, J. Maillard, A. Sun, S. Wang, G. Wenzek, and A. Youngblood. Scaling Neural Machine Translation to 200 Languages. *Nature* 630, 8018 (2024), 841–846. https://doi.org/10.1038/s41586-024-07335-x

[22] Y. Qian and J. Lehman. Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review. *ACM Transactions on Computing Education* 18, 1 (2017), Article 1, 24 pages. https://doi.org/10.1145/3077618

[23] R. Scherer, F. Siddiq, and B. Sánchez Viveros. A Meta-Analysis of Teaching and Learning Computer Programming: Effective Instructional Approaches and Conditions. *Computers in Human Behavior* 109 (2020), Article 106349. https://doi.org/10.1016/j.chb.2020.106349

[24] I. Sommerville. *Software Engineering*. 10th ed. Pearson, Boston, MA, USA, 2016.

[25] J. Sorva. Notional Machines and Introductory Programming Education. *ACM Transactions on Computing Education* 13, 2 (2013), Article 8, 31 pages. https://doi.org/10.1145/2483710.2483713

[26] J. Sorva, V. Karavirta, and L. Malmi. A Review of Generic Program Visualization Systems for Introductory Programming Education. *ACM Transactions on Computing Education* 13, 4 (2013), Article 15, 64 pages. https://doi.org/10.1145/2490822

[27] A. Swidan and F. Hermans. A Framework for the Localization of Programming Languages. In *Proc. SPLASH-E 2023*. ACM, New York, 13–25. https://doi.org/10.1145/3622780.3623645

[28] University of Cape Town. *Intellectual Property Policy*. University of Cape Town, Cape Town, South Africa, 2011. https://uct.ac.za/sites/default/files/media/documents/uct_ac_za/190/Policy_Intellectual_Property_2011.pdf

[29] University of Cape Town. *UCT Language Policy*. University of Cape Town, Cape Town, South Africa. Approved by Council, 12 December 2024. https://uct.ac.za/sites/default/files/media/documents/policy-lpc-language-december-2024-english.pdf

[30] V. Vainio and J. Sajaniemi. Factors in Novice Programmers' Poor Tracing Skills. In *Proc. ITiCSE 2007*. ACM, New York, 236–240. https://doi.org/10.1145/1268784.1268853

[31] C. Watson and F. W. B. Li. Failure Rates in Introductory Programming Revisited. In *Proc. ITiCSE 2014*. ACM, New York, 39–44. https://doi.org/10.1145/2591708.2591749

Supplementary Information

A Misconception Catalogue

Table A.1: Misconception catalogue and source traceability.

Identifier	Misconception	Source
`strings_need_quotes`	Unquoted text is a string value	Kohn [14]
`string_vs_list_confusion`	Bracketed characters form a string	Local
`escape_sequence_literal_confusion`	`\n` is two literal characters	Curriculum
`concatenation_adds_space`	`+` inserts a separator	Local
`concatenation_requires_same_type_fear`	Strings must match in kind to join	Qian and Lehman [22]
`repetition_confused_with_multiplication`	`*` performs arithmetic	Qian and Lehman [22]
`indexing_from_one`	The first character is at index 1	Izu and Weerasinghe [12]; curriculum
`negative_index_confusion`	Negative indices are invalid or count forward	Local
`off_by_one_length`	`len(s)` is a valid index	Vainio and Sajaniemi [30]; local
`slice_stop_inclusive`	The stop bound is included	Luxton-Reilly and Petersen [16]; local
`negative_step_unfamiliar`	A negative step is an error	Local
`immutability_in_place`	Operations modify the original string	Lu and Krishnamurthi [15]
`method_mutates_target`	Methods change the variable	Lu and Krishnamurthi [15]; Izu and Weerasinghe [12]
`strip_only_removes_one_char`	`strip()` removes a single character	Local
`count_overlap_confusion`	`count()` counts overlapping matches	Local
`count_vs_find_confusion`	`count()` returns a position	Izu and Weerasinghe [12]
`comparison_ignores_case`	`==` is case-insensitive	Altadmri and Brown [2]; Blok and Fehnker [3]
`strings_not_orderable_myth`	Strings cannot be ordered	Local
`string_number_equality_myth`	`'10' == 10`	Qian and Lehman [22]
`fstring_missing_prefix_effect`	Braces substitute without the `f` prefix	Hughes et al. [11]
`slice_defaults` (concept)	An omitted bound falls back to its default	Curriculum
`slice_step` (concept)	Step selects within the slice rather than moving the bounds	Curriculum
`slice_clamping` (concept)	Slice bounds clamp; indexing does not	Curriculum
`slice_empty_result` (concept)	A slice that cannot advance returns empty	Curriculum

B Full Requirements Specification

Table B.1: Functional requirements and measurable acceptance conditions.

ID	Requirement	Acceptance condition
FR-1	The system should present string topics as learning modules with step-through worked examples, including indexing, slicing, and immutability.	All eight modules are available and each presents the specified staged learning content.
FR-2	The visualiser should display a string as indexed characters and show the effect of string operations.	Reference index and slice expressions reproduce the corresponding Python results in the visualiser.
FR-3	The visualiser should provide play, pause, and speed controls.	Each control changes visualiser playback as intended.
FR-4	The visualiser should be usable independently of the modules, including by guest users.	A guest can open and use the standalone visualiser without an active module or account.
FR-5	Code Space should allow learners to write and run Python independently of the lessons and should display output and errors.	A guest can submit valid and invalid code and receive the corresponding output or error.
FR-6	Each module should include graded practice exercises.	Each module exposes graded practice using the supported question types.
FR-7	Where a wrong answer maps to a known string misconception, the system should return feedback for that misconception.	A diagnosed misconception returns the corresponding feedback explanation.
FR-8	Lesson copy, interface text, and exercise feedback should be available in English and isiXhosa.	Supported lesson, interface, and feedback content can be rendered in both locales.
FR-9	Learners should be able to switch between English and isiXhosa without losing their place.	Changing locale preserves the learner's current task and position.
FR-10	When the UI language is isiXhosa, displayed snippets may use isiXhosa keywords; code sent to the back end for execution or grading should be Python.	Localised code is translated before execution and executable requests contain standard Python.
FR-11	Guests may use the Visualiser, Code Space, Docs, and Settings. Modules, Challenges, and the leaderboard should require a non-guest account.	Guest and authenticated access restrictions match the specified feature set.
FR-12	Signed-in learners should have progress tracking, Challenges, a leaderboard, and guidance on using the modules and visualiser.	Authenticated users can access the specified progress and supplementary-learning features.
FR-13	The visualiser should offer spoken narration that describes each step, explains why the result follows, and on failure names the error and the reason it occurred.	Narration explains both the action and its reason for successful and failed evaluations.
FR-14	Spoken narration should complete its current sentence before the visualisation advances to the next step.	The next visualisation step does not begin until the utterance completes.
FR-15	Where spoken narration is unavailable in the selected language, the system should explain the limitation rather than disabling the control silently.	Selecting unavailable narration produces an explicit limitation message.
FR-16	The output area should show a distinct visual state when the result is an error.	Error output is visually distinguishable from successful output.
FR-17	A lecturer account should be able to view aggregate and per-student progress, marks, and misconception frequency across the cohort.	A lecturer-role account can reach the roster, per-student breakdown, and marks view; a non-lecturer account cannot.
FR-18	A lecturer account should be able to create, edit, reorder, and publish or hide modules and exercises through the lecturer dashboard.	A lecturer can create or modify modules and exercises, change ordering, publish or hide them, and subsequent learner views reflect the resulting content; non-lecturers cannot perform these operations.
FR-19	Lecturer-facing views should distinguish a value that has not yet been recorded from a value of zero.	A student/module with no attempts renders as unrecorded rather than zero.

Table B.2: Non-functional requirements and measurable acceptance conditions.

ID	Requirement	Acceptance condition
NFR-1	A learner should be able to use the system's interface and operations without confusion or assistance.	Target users can complete the defined tasks without researcher assistance except where they cannot otherwise proceed.
NFR-2	The application should run in a web browser without installation.	The deployed application is accessible through a supported web browser without local installation.
NFR-3	The visualiser and Code Space should update without noticeable delay after learner input.	Normal interaction produces no reported delay that prevents completion of the task.
NFR-4	The system should complete a session without crashing.	The system remains operational throughout a complete evaluation session without an application crash.
NFR-5	The interface should remain consistent across modules and when the language or theme is changed.	Navigation, controls, and layout remain functionally consistent after module, language, and theme changes.
NFR-6	The system should work in common browsers and on different screen sizes.	The deployed interface loads and remains usable in the supported browser and viewport configurations used for testing.
NFR-7	Changing language should not require leaving the current task or losing progress within it.	Locale switching preserves the learner's current task and local progress state.
NFR-8	Adding a new module or language should not require changes to existing components.	New content or locale data can be introduced through the existing data and dispatch structures without modifying shared renderers.
NFR-9	The front end should not grade or execute untrusted learner code for a mark; it should send that code to the back end.	No answer key or grading decision is performed in the browser; grading and execution requests are sent to the back end.
NFR-10	No lesson field or interface string should render blank when its isiXhosa translation is missing.	An absent isiXhosa translation falls back to readable English content rather than an empty field.
NFR-11	Animation and narration timing should be driven by speech completion rather than a fixed interval, so no spoken explanation is cut off.	Visual progression waits for speech completion before advancing.
NFR-12	Visualiser state must be exposed to assistive technology and no information may depend on colour alone.	Slice boundaries and state remain understandable through labels and accessible state information without relying solely on colour.
NFR-13	The front end must not compute a student's mark for lecturer display; it must render whatever the back end reports.	The dashboard performs no client-side aggregation, weighting, or pass/fail calculation and renders back-end-supplied figures directly.

C System Architecture and Sequence Diagrams

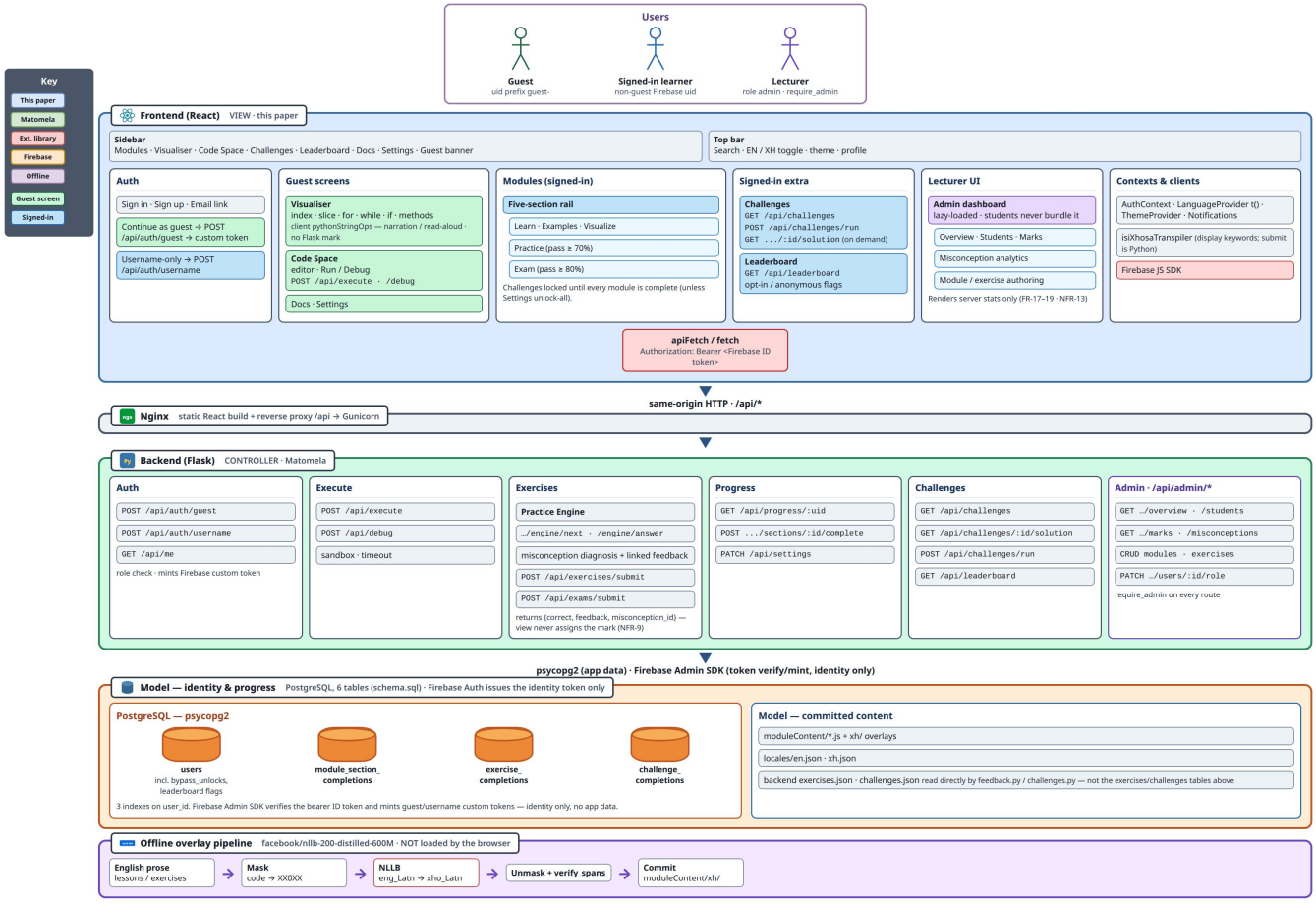


Figure 1. System architecture of String Conundrum (logical MVC).
Guest: Visualiser (incl. narration), Code Space, Docs, Settings. Signed-in: Modules, Challenges, Leaderboard. Lecturers: admin dashboard and /api/admin/*. Practice Engine provides procedural generation and misconception diagnosis. The React view never assigns a mark.

Figure C.1: System architecture of String Conundrum, showing the endpoint-level structure and the responsibility boundary between front-end and back-end components.

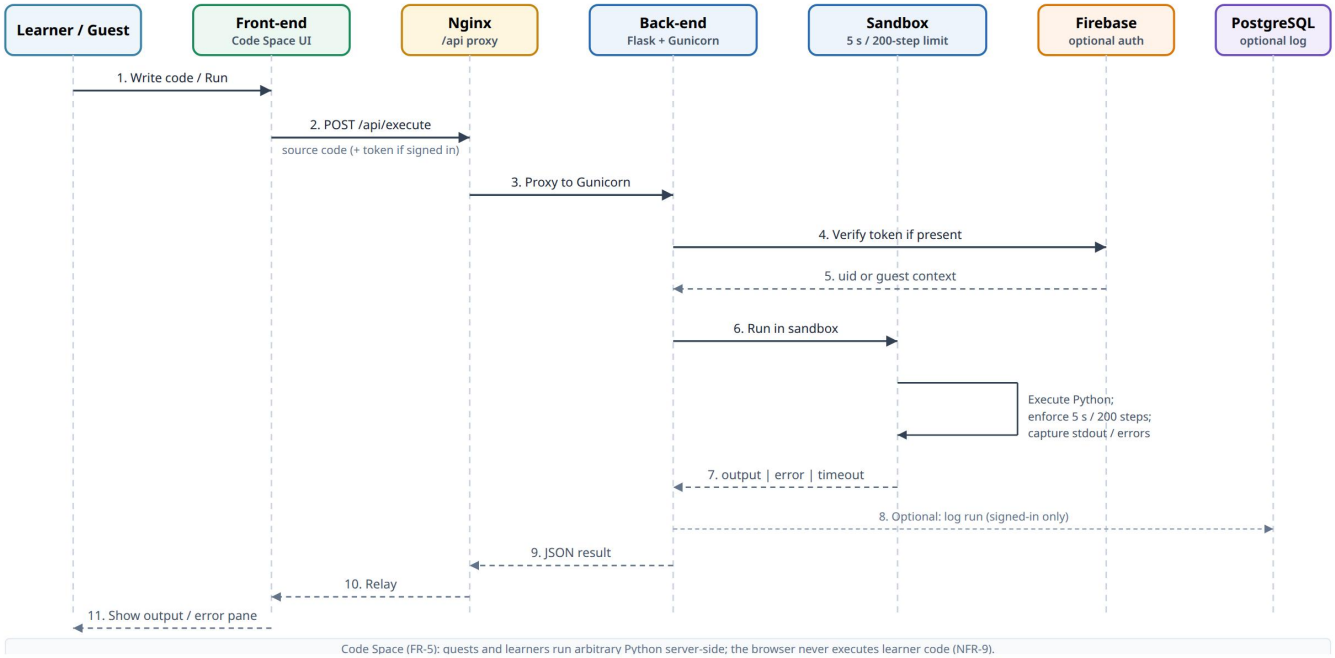


Figure C.2: Code Space execution sequence. The learner submits Python for server-side execution and receives output, errors, or a timeout.

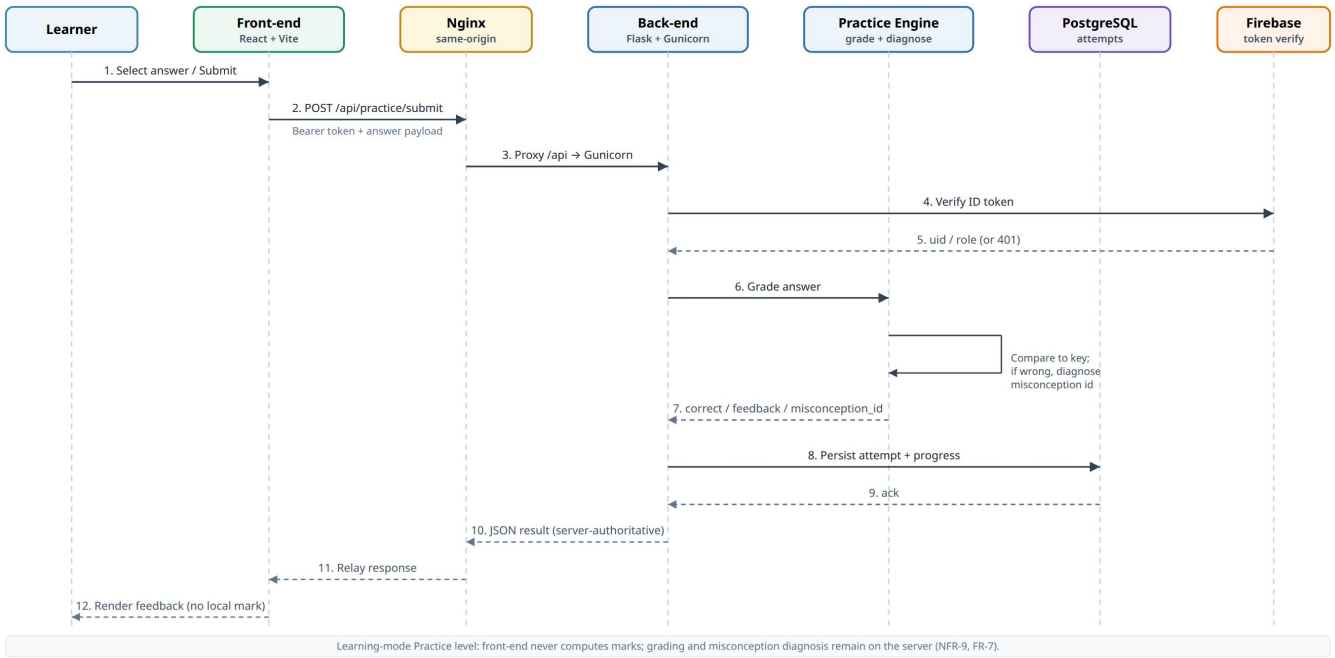


Figure C.3: Sequence of a graded practice submission across the front end, proxy, back end, Practice Engine, and persistence services.

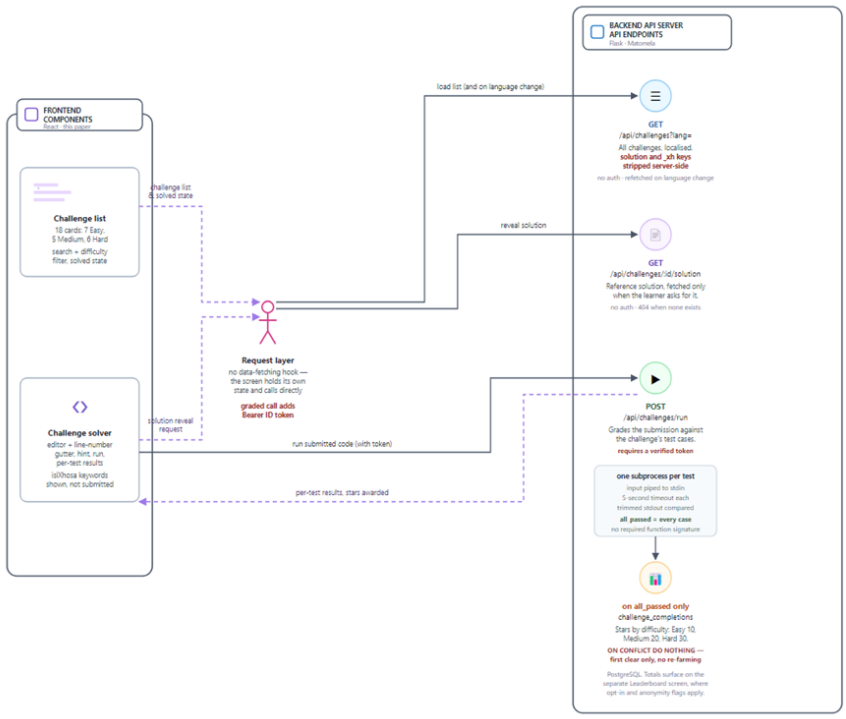


Figure. Challenges: frontend components, request layer, and backend endpoints.
Solid lines are calls out; dashed lines are data returning. Challenges is gated on completing all eight modules (or the unlock-all setting), so it is unreachable for guests. The view never decides a pass — test-case comparison and star awards happen server-side (NFR-9).

Figure C.4: Challenges. Frontend components, request layer, and backend endpoints.

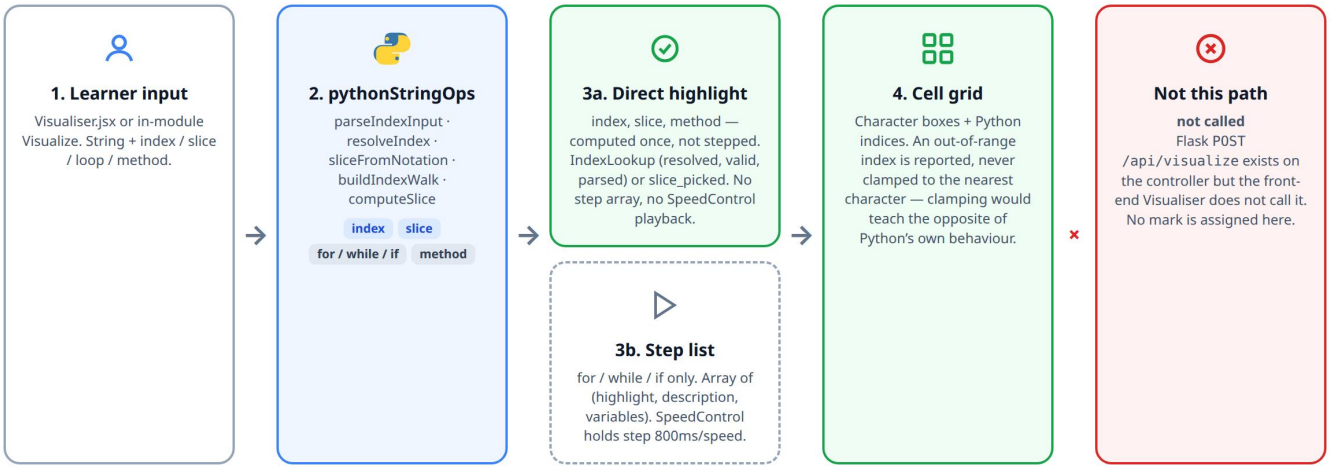
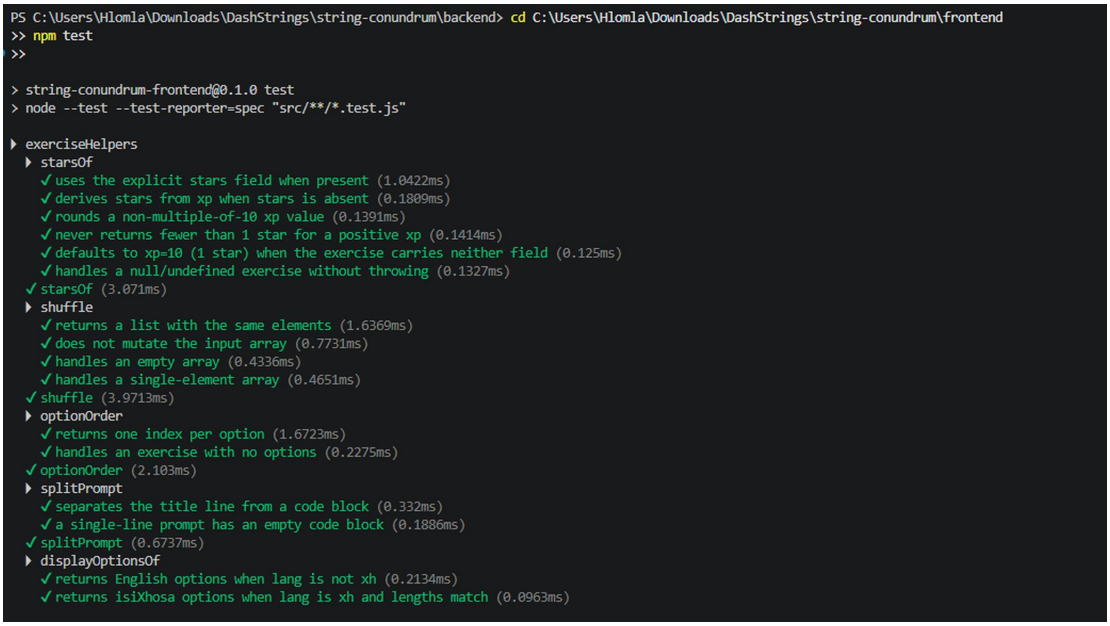


Figure C.5: Sequence of operations in the visualiser evaluation pipeline.

D Correctness and Development Testing

Table D.1: Front-end unit tests by area, including reference cases where applicable.

Area under test	Tests	Covered behaviour	Requirements
String slice and index engine	11	Index parsing, index resolution, index walks, slice parsing, slice notation, and slice computation. An out-of-range index is reported as invalid while out-of-range slice bounds are clamped according to Python slicing semantics, and a step of zero reports Python's own `ValueError`, all checked against CPython behaviour. The reference set contains 60 index expressions and 132 slice expressions.	FR-2
Shared practice and examination helpers	25	Star awards, option shuffling and ordering, prompt splitting, localised option fallback, and default answer shaping across the four question types	FR-6, FR-7
Account credential validation	24	Email shape, UCT address, username normalisation and validation, and Firebase error-code mapping applied before account creation	FR-11, FR-12
Password policy	15	Strength scoring, policy enforcement, and individual rule definitions at sign-up	FR-11
isiXhosa keyword transpiler	10	Keyword substitution in both directions while leaving keywords inside string literals and comments untouched	FR-10
isiXhosa content merge	16	Field-wise overlay of isiXhosa content over the English base, retaining English wherever a translation is absent	FR-8, NFR-10
Module stage sequence and gating	20	Five-stage sequence, examination gating, and fallback behaviour for an unrecognised module	FR-1, FR-12
Syntax highlighting	12	Keyword and builtin highlighting of isiXhosa-form Python	FR-10
Total	**133**		



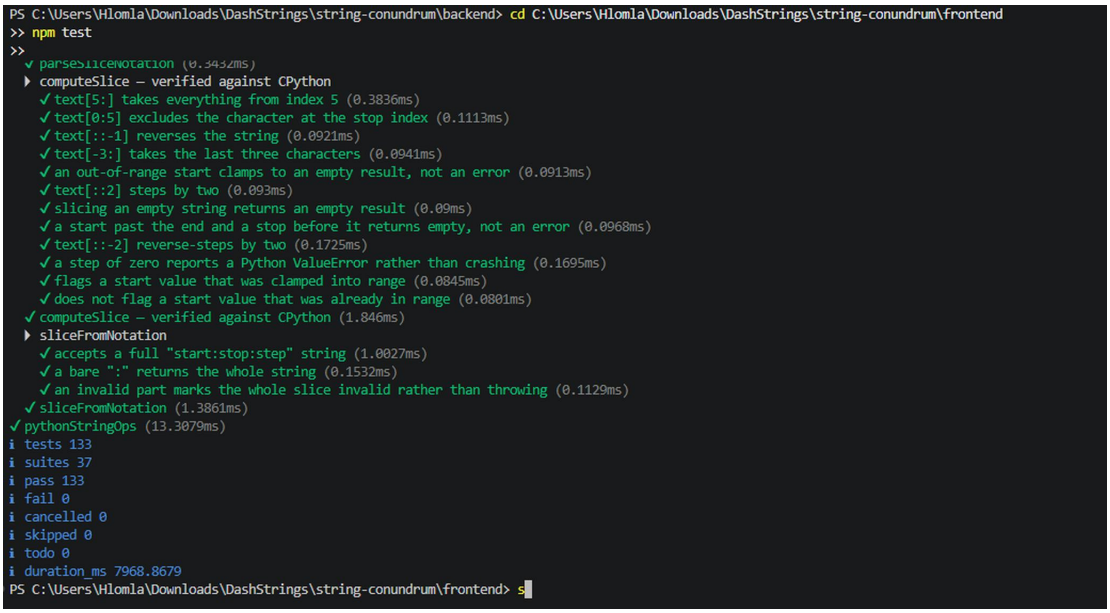


Figure D.1: Front-end test suite execution. The run begins with the shared exercise helpers and ends with the string slice and index engine, with all 133 tests across 37 suites passing and no failures, cancellations, or skipped cases.

E Offline NLLB-200 Overlay Generation Pipeline



Figure E.1: Offline NLLB-200 overlay generation.

F Evaluation Materials

F.1 Participant Post-Session Booklet

Table F.1: Participant post-session booklet structure.

Part	Content
A	Retrospective confidence ratings for indexing, slicing, immutability, and comparison, plus the rating of whether the isiXhosa content assisted understanding.
B	Applied knowledge check covering indexing, slicing, immutability, and comparison. Responses were marked against expected answers.
C	Open reflection on what became clearer and what remained unclear.
D	Feedback on explanations, interface behaviour, confusing elements, and suggested improvements.
E	Ten-item System Usability Scale using a five-point agreement scale.

F.2 Usability Tasks and Observation Criteria

Table F.2: Usability task areas and observation criteria.

Task area	Observation criterion
Module learning	Participant can navigate the selected module, interpret the staged examples, and progress through its learning stages.
Visualiser	Participant can use the visualiser to inspect indexing, slicing, and operation results.
Practice and examination	Participant can answer graded questions and interpret the resulting feedback.
Code Space	Participant can write and execute Python independently of the module sequence.
Challenges and leaderboard	Participant can access and use the supplementary practice and progress features where available.
Navigation and localisation	Participant can move between relevant areas and use the language controls without losing context.

F.3 User Acceptance Testing Checklist

Verdict key: Accepted = requirement met; Accepted with comment = requirement met with a recorded comment; Requires fix = correction required before submission; Not applicable = not applicable to the test session; Not recorded = no verdict was recorded.

Table F.3: Final user acceptance testing checklist.

Feature	Verdict	Comment
Account creation and sign-in	Accepted	
Guest access to Code Space, Visualiser, Docs, and Settings	Accepted	
Sign-out	Accepted	
Eight learning modules	Accepted	Module visuals were described as good and easy to follow.
Code Space	Accepted	Clear-editor control suggested as an improvement.
Standalone Visualiser	Accepted	
Step-through execution tracer	Accepted	
Multiple-choice grading	Accepted	
Select-all grading	Accepted	
Short-answer grading	Accepted	
Code-writing grading	Accepted	
Misconception-linked feedback	Accepted	
English/isiXhosa localisation	Accepted	
Challenges	Accepted	Described as a welcome addition.
Leaderboard	Accepted	Described as a welcome addition.
Settings	Accepted	
Navigation	Accepted	Recorded as working.
Production deployment	Accepted	
Normal-use links and console behaviour	Accepted	
Sign-up by email link	Not recorded	No verdict recorded.
Escape-sequence rendering	Requires fix	An escape sequence such as `\n` was shown as two literal characters during UAT. The rendering issue was corrected before final submission.

F.4 Peer Developer Evaluation Instrument

Table F.4: Peer developer evaluation questions.

Question	Focus
1	First impressions of the system.
2	What worked well.
3	What the reviewer would question or build differently.
4	How the reviewer's own system addresses the same problem differently.
5	Anything confusing, broken, or unclear.
6	Overall assessment of the system.

G Raw Results and Per-Question Breakdowns

G.1 System Usability Scale Results

Table G.1: System Usability Scale summary by usability cycle.

Cycle	Participants	Complete SUS	Mean	Median
Cycle 1	11	10	79.2	82.5
Cycle 2	11	10	82.5	88.75
Cycle 3	13	13	79.8	80.0

G.2 Confidence Ratings

Table G.2: Mean retrospective confidence ratings before and after Cycles 1 and 2.

Concept	C1 Before	C1 After	C2 Before	C2 After
String indexing	3.36	4.50	3.64	4.27
String slicing	3.18	4.36	3.36	4.55
String immutability	2.70	4.50	2.82	4.45
String comparison	3.11	4.60	3.18	4.45
String looping	–	–	3.00	4.67

G.3 Applied Knowledge Check

Table G.3: Applied knowledge-check outcomes by concept for Cycles 1 and 2.

	Cycle 1			Cycle 2		
Concept	Full	Partial	Incorrect	Full	Partial	Incorrect
Indexing	10	0	1	11	0	0
Slicing	9	0	2	11	0	0
Comparison	11	0	0	11	0	0
Immutability	5	3	3	7	1	3

G.4 Applied Knowledge Uncertainty

Table G.4: Fully correct applied-knowledge responses with 95% Wilson confidence intervals.

Concept	Cycle 1	Cycle 2	Cycle 3
Indexing	10/11 [62.3, 98.4]	11/11 [74.1, 100.0]	12/13 [66.7, 98.6]
Slicing	9/11 [52.3, 94.9]	11/11 [74.1, 100.0]	10/13 [49.7, 91.8]
Comparison	11/11 [74.1, 100.0]	11/11 [74.1, 100.0]	10/13 [49.7, 91.8]
Immutability	5/11 [21.3, 72.0]	7/11 [35.4, 84.8]	8/13 [35.5, 82.3]
Method mutation	–	–	11/13 [57.8, 95.7]

Values in brackets are percentage-point confidence intervals. Their width reflects the small sample sizes and limits precise between-cycle comparison.

G.5 Peer Developer Evaluation Findings

Table G.5: Recurring themes in the three peer developer evaluations.

Theme	Finding
Visualiser	Named positively by all three reviewers and described as interactive, intuitive, and a strong component of the system.
Interface and modules	Reviewers described the interface as clean and the lesson structure as thorough and informative.
Examination bank	All three reviewers identified problems including repetitive questions, incorrect marking, answer-option defects, and insufficient treatment of negative indexing.
Challenge feedback	One reviewer suggested that challenge errors should explain the cause rather than merely expose the returned error.
Overall impression	All three closing summaries were positive.

G.6 User Acceptance Testing Outcome

Table G.6: Summary of the final user acceptance testing outcome.

Outcome	Recorded result
Overall acceptance	The system was accepted against the project requirements.
Accepted feature areas	Account and access; all eight modules; Code Space; the Visualiser; execution tracing; four grading types; misconception-linked feedback; isiXhosa localisation; Challenges; leaderboard; Settings; navigation; deployment; and normal-use link and console checks.
Improvement requested	Add an option to clear the Code Space contents.
Defect identified during UAT	An escape sequence such as `\n` was shown as two literal characters during UAT. This was corrected before final submission.
Supervisor assessment	The system was described as easy to use, helpful, and well presented.

H Cycle 3 Evaluation Results

H.1 Participant and Module Summary

Table H.1: Cycle 3 participant and module-completion summary.

Measure	Result
Total participants	13
Participants enrolled in CSC1016S	13
Mainstream first-year participants	10
Extended-programme CSC1016S participants	3
Completed introductory Python	13
Introduction to Strings completed	7
Slicing completed	10
Looping completed	6

H.2 Cycle 3 Confidence

Table H.2: Cycle 3 mean retrospective confidence ratings.

Concept	Before	After	Change
Indexing	3.38	4.38	+1.00
Slicing	3.23	4.23	+1.00
Looping	3.15	4.31	+1.15
Comparison	2.69	3.77	+1.08
Tracing code	2.85	4.08	+1.23

H.3 Cycle 3 Applied Knowledge

Table H.3: Cycle 3 applied-knowledge results.

Question	Correct	Percentage
Q1 Indexing	12/13	92.3%
Q2 Slicing	10/13	76.9%
Q3 Immutability	8/13	61.5%
Q4 `upper()` and original string	11/13	84.6%
Q5 Comparison	10/13	76.9%
All five correct	3/13	23.1%

H.4 Cycle 3 Most Helpful Features

Table H.4: Feature most frequently identified as helpful in Cycle 3.

Feature	n	Percentage
Visualiser	5	38.5%
Practice questions	3	23.1%
Feedback/explanations	2	15.4%
Challenges	2	15.4%
Code Space	1	7.7%

H.5 Cycle 3 Reported Issues

Table H.5: Cycle 3 reported issues. Multiple issues could be selected by one participant.

Issue	n	Percentage
No major issues	5	38.5%
Unsure what to do next	5	38.5%
Session too long	4	30.8%
Did not trust grading	3	23.1%
isiXhosa terms unclear/inconsistent	3	23.1%
Feedback unclear	2	15.4%
Navigation confusing	1	7.7%
Code Space confusing	1	7.7%
Challenges difficult/unclear	1	7.7%

H.6 Cycle 3 Localisation

Table H.6: Cycle 3 localisation ratings.

Measure	n	Mean	Median
isiXhosa content helpfulness	10	3.70	4
Translation rating	13	3.31	3
isiXhosa transpiler	13	3.46	4

H.7 Cycle 3 Questionnaire

The Cycle 3 instrument extended the earlier booklet with confidence ratings for looping and tracing, ratings of specific platform features, a mental-demand item, a feature-use record, an expanded issue checklist, and five applied knowledge questions. The exact administered questionnaire should be retained alongside the anonymised data as the primary instrument record. Its five knowledge questions were: (1) `word = "Python"; print(word[1])`; (2) `word = "Python"; print(word[1:4])`; (3) `word = "Python"; word[0] = "J"`; (4) `text = "hello"; print(text.upper()); print(text)`; and (5) `first = "hello"; second = "Hello"; print(first == second)`.

H.8 Analysis Notes

Percentages in the Cycle 3 tables use 13 participants as the denominator unless otherwise stated. The isiXhosa helpfulness item was answered by 10 participants and is therefore reported with its own denominator. Multiple issue categories could be selected by one participant, so issue percentages are not expected to sum to 100%. The Cycle 3 knowledge instrument differs from the earlier four-question instrument, so its values are reported descriptively rather than used as a strict longitudinal test.

I Feedback-Related Changes

Table I.1: Changes made in response to Cycle 1 evaluation findings.

Finding	Change	Purpose
Need for audio support	Added spoken narration using browser speech synthesis	Improve accessibility and provide an additional explanation mode.
Narration stated what happened without why	Reworked closing and error narration to state the reason	Make feedback explanatory rather than merely descriptive.
Unclear module completion threshold	Added explicit pass-or-retry feedback around the 80% threshold	Make progression requirements visible.
Weak areas ended as a list	Linked identified weak areas back into practice	Provide a direct remediation path.
Expected examination output was unclear	Replaced wrapping output chips with labelled rows	Make practical-answer expectations easier to interpret.
Module navigation returned no overview	Made the module navigation control return to the overview	Remove a navigation dead end.
Learners requested freer module access	Surfaced the existing all-module unlock setting at the restriction	Preserve deliberate gating while making the option visible.
Correct answer often appeared in the same position	Shuffled answer options	Reduce answer-position cues.
Visualiser required a manual trigger	Added automatic playback on entering the Visualise stage	Reduce discovery friction.
Challenges lacked line numbers	Added a line-number gutter	Align the Challenges editor with Code Space.
Failed visualiser evaluations produced insufficient feedback	Changed failures to report an error rather than fabricate a result	Prevent invalid expressions from appearing to produce valid output.

Table I.2: Cycle 2 findings retained as further refinement or future development work.

Finding	Suggested refinement
String length was not visible without the browser console	Display string or character length directly in the relevant learning or visualisation context.
Keyword reference lacked fuller definitions	Add hover definitions or expanded explanations for isiXhosa keyword entries.
Dark-theme text contrast remained an issue	Review the contrast of explanatory and status text.
Visualiser could continue after a syntax error	Stop visualisation when the source code is syntactically invalid.
Some slicing explanations were ambiguous	Replace the confusing analogy with a direct explanation of start, stop, and step semantics.
Practice could feel repetitive	Expand the range of concepts and edge cases represented within individual modules.
Requests for broader Python concepts	Treat broader coverage as project scope for future extension.

I.1 Cycle 3 Feature Agreement Ratings

Table I.3: Cycle 3 mean agreement ratings on the revised feature scale.

Statement	Mean
Index and slice highlighting helped reasoning	4.69
Visualiser made code behaviour easier to understand	4.46
Feedback after wrong answers corrected misconceptions	4.46
Distinguishing mutation from reassignment	4.15
Code Space helped experimentation	4.31
Challenges tested understanding	4.23
Practice questions felt relevant	3.85
isiXhosa transpiler worked as expected	3.46
Leaderboard increased motivation	3.54
isiXhosa translation made content easier	3.31

J Supplementary Figures

The following figures provide additional detail supporting the Results section. They are not required for understanding the main claims and are therefore placed here so that the ten-page main-paper limit is preserved.

J.1 Retrospective Confidence Ratings

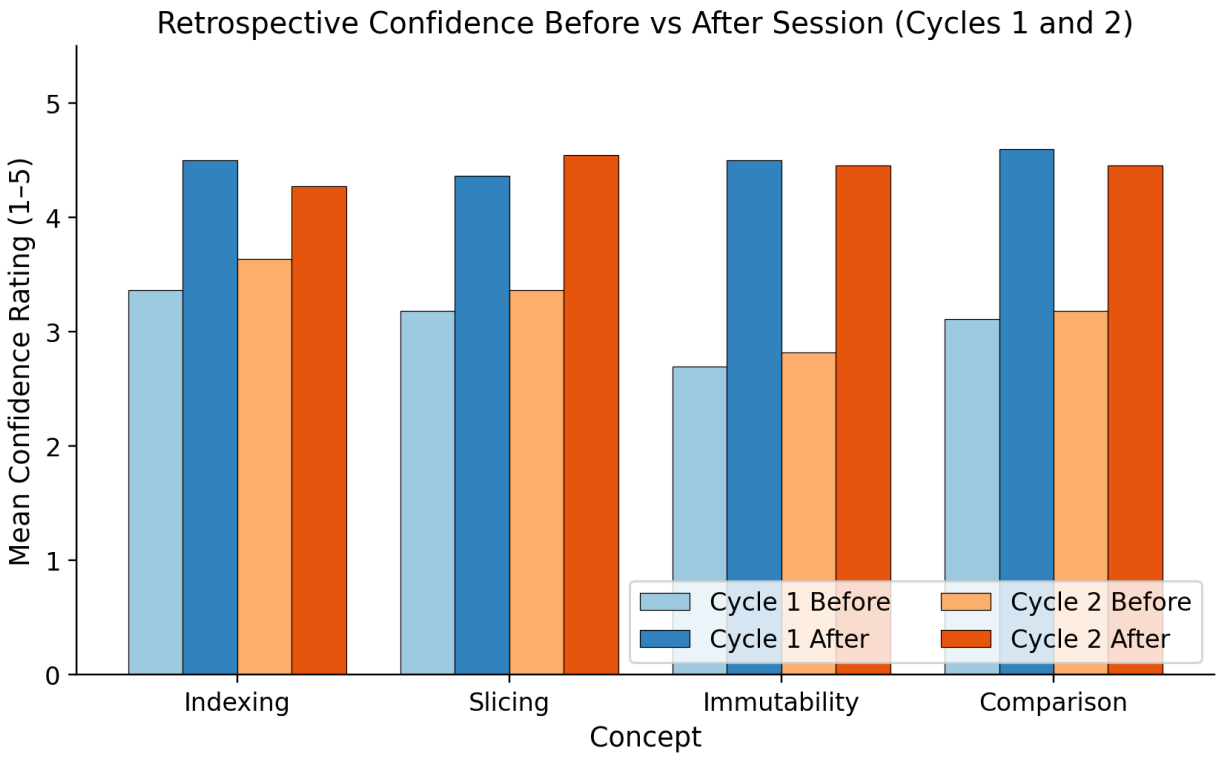


Figure J.1: Mean retrospective confidence ratings before and after the session for Cycles 1 and 2 (1–5 scale).

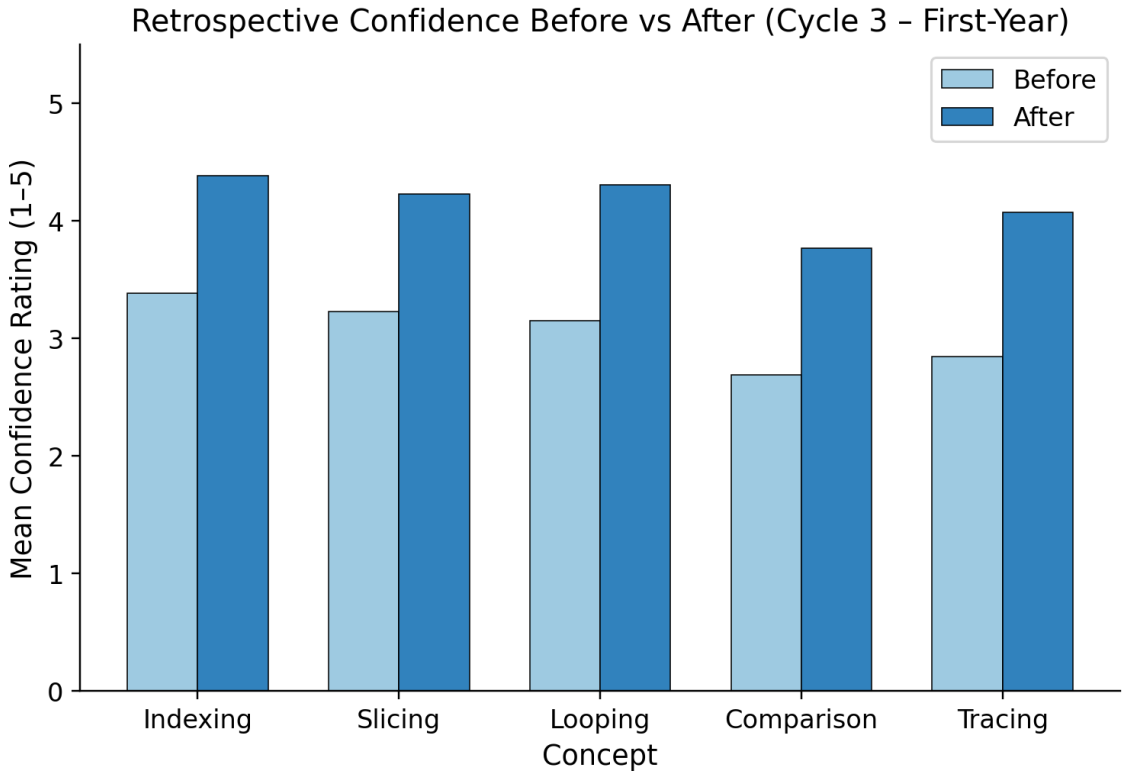


Figure J.2: Mean retrospective confidence ratings before and after the session for Cycle 3 (CSC1016S cohort). Concepts include indexing, slicing, looping, comparison, and tracing code step-by-step.

J.2 isiXhosa Content Helpfulness

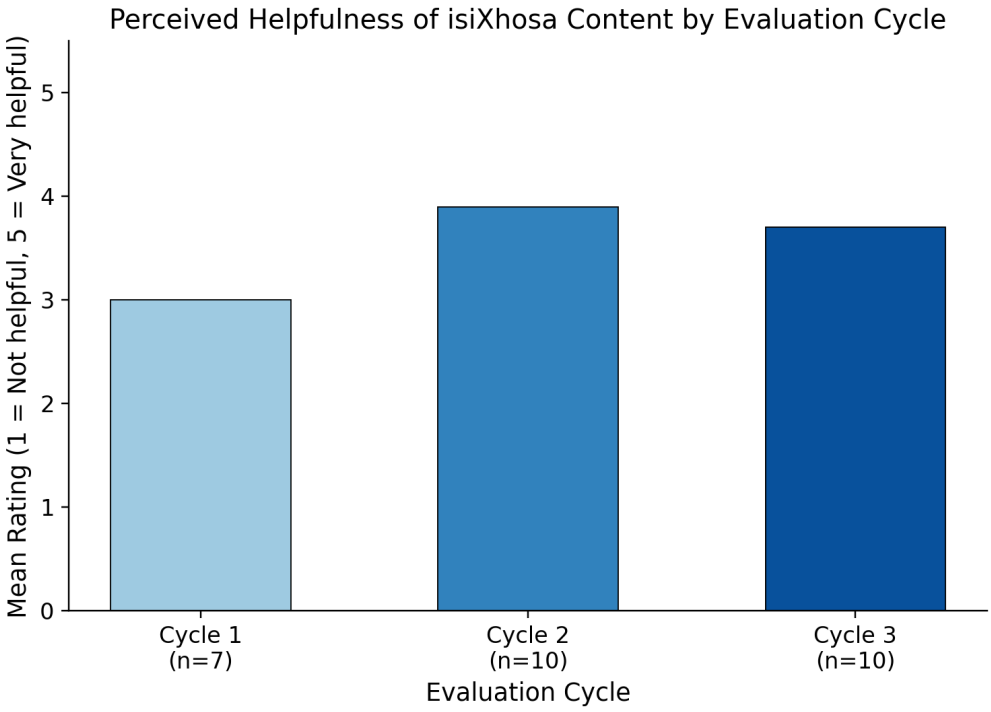


Figure J.3: Mean perceived helpfulness of isiXhosa content by evaluation cycle (1 = not helpful, 5 = very helpful). Only participants who answered the item are included.

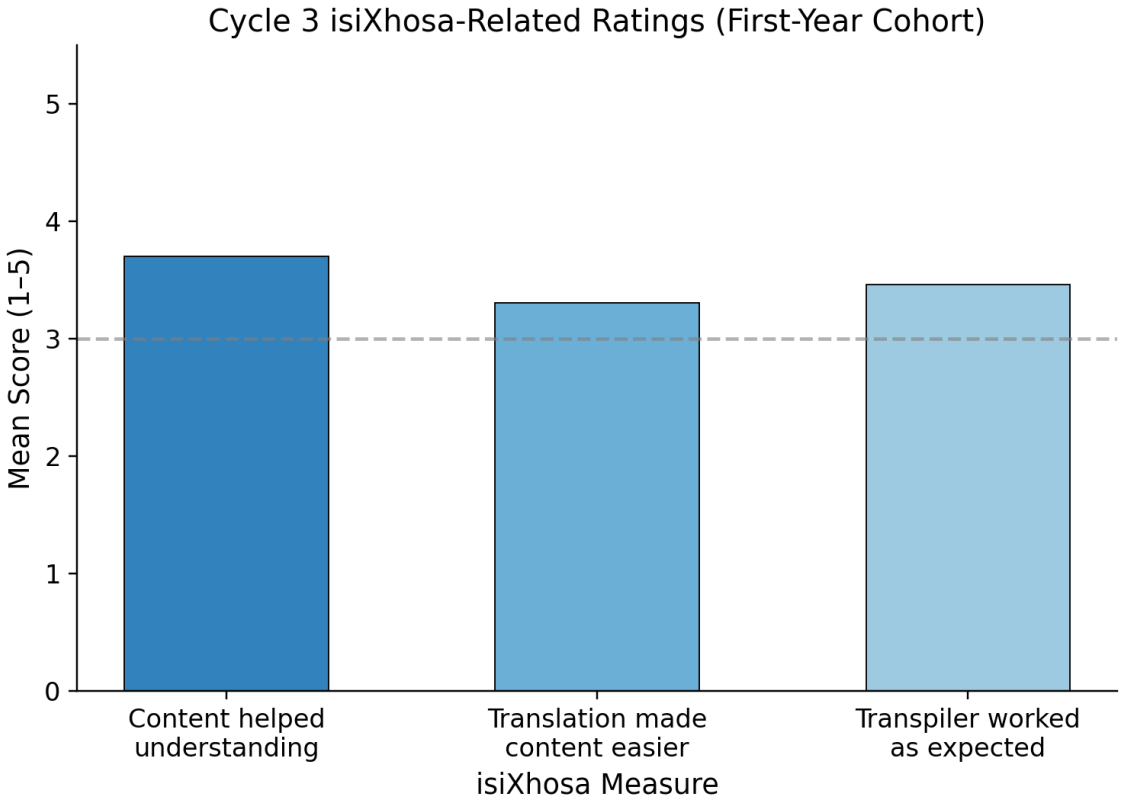


Figure J.4: Cycle 3 detailed isiXhosa-related ratings for perceived helpfulness of content, agreement that translation made content easier, and agreement that the transpiler worked as expected (1–5 scale).

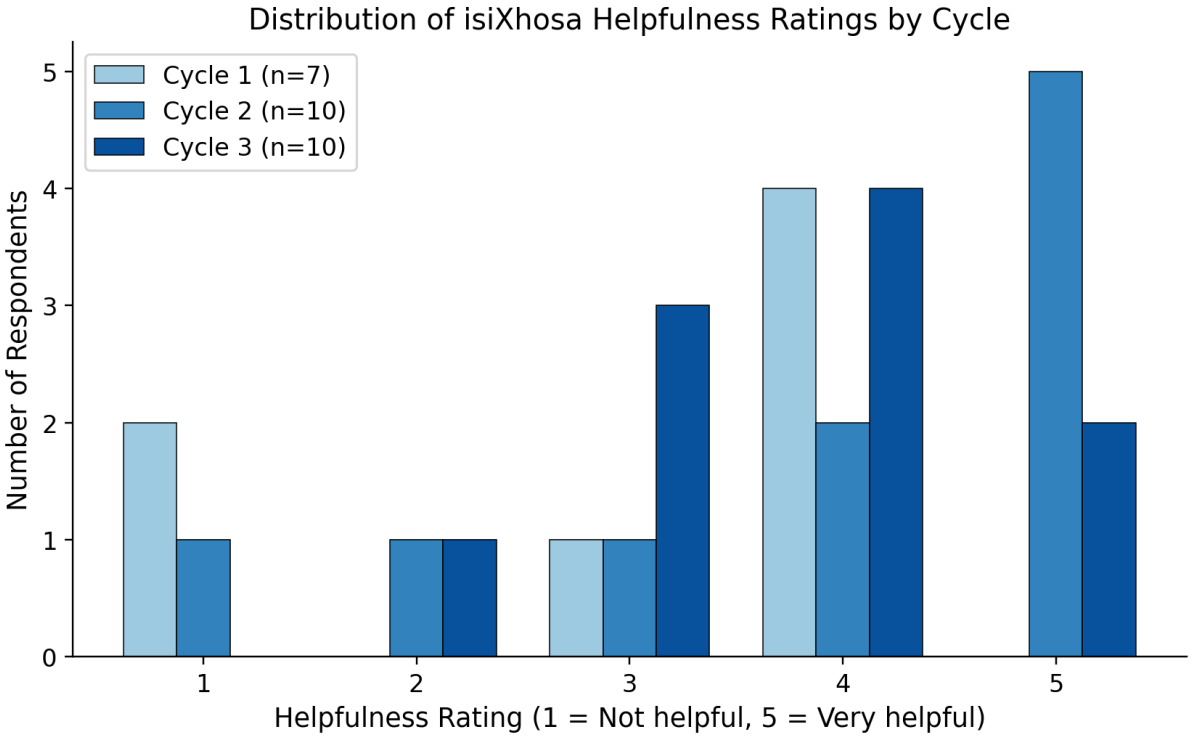


Figure J.5: Distribution of isiXhosa helpfulness ratings (1 = not helpful, 5 = very helpful) across the three evaluation cycles. Only respondents who answered the item are shown.

J.3 Alternative SUS Visualisation

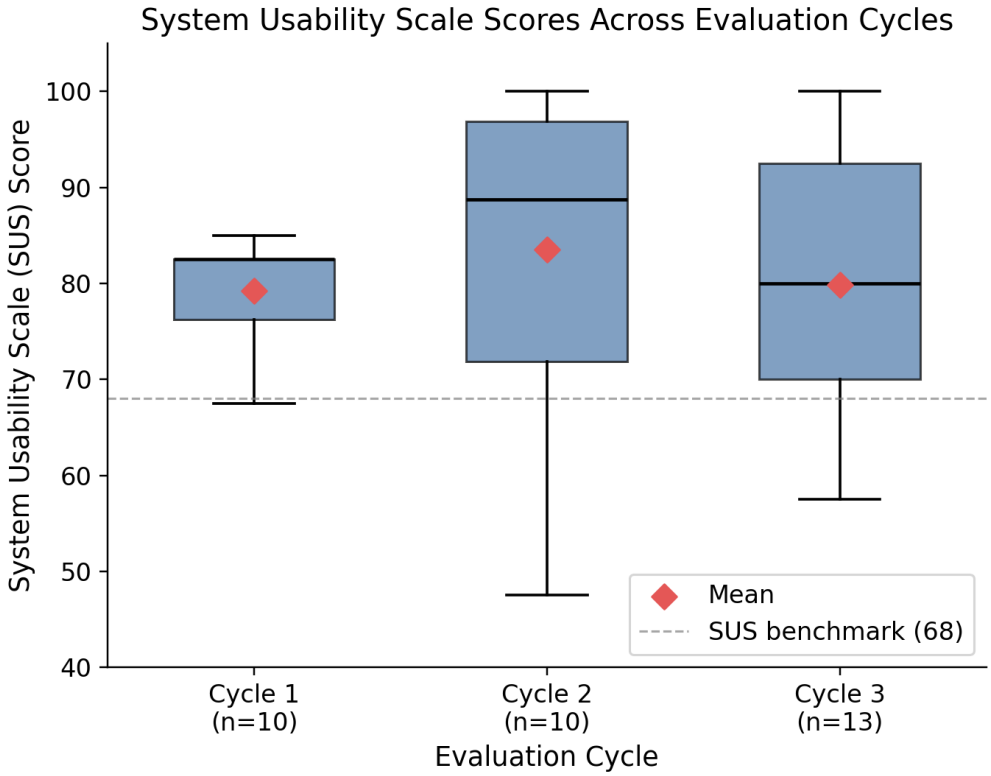


Figure J.6: Box-plot distribution of System Usability Scale scores across the three evaluation cycles. Diamonds mark the mean; the dashed line indicates the conventional SUS benchmark of 68.

K Evaluation Cycle Conditions and Decision Records

K.1 Evaluation Cycle Conditions

Table K.1: Evaluation conditions across the three target-user cycles.

Condition	Cycle 1	Cycle 2	Cycle 3
Participants	11	11	13
Academic level	Second and third year	Second and third year	CSC1016S cohort
Evaluation context	Target-user session	Target-user session	Immediately after CSC1016S Java practical
Room and session	Planned evaluation setting	Planned evaluation setting	Same room and same scheduled session as the practical
Session duration	Approximately 30 minutes	Approximately 30 minutes	Approximately 30 minutes
Module exposure	Two of eight modules	Two of eight modules	Two of eight modules
Researcher present	Yes	Yes	Yes
SUS instrument	Unchanged	Unchanged	Unchanged
Confidence instrument	Original	Original	Revised
Knowledge instrument	Original	Original	Revised with method-mutation question
Agreement instrument	Original	Original	Expanded feature-level ratings

K.2 Evaluation Method Rationale

Table K.2: Rationale and limitations for each evaluation method.

Method	Evidence source	Primary purpose	Main limitation
Target-user usability	Undergraduate students	Usability, engagement, and demonstrated understanding during interaction	Small non-equivalent cohorts and short sessions
Expert interface review	Four HCI postgraduate students	Qualitative interface review independent of learner content expertise	No structured numerical scoring instrument
Peer developer evaluation	Three PyTutor developers	Independent technical perspective on design and implementation choices	Small specialist sample and open-response instrument
User acceptance testing	Project supervisor	Requirement satisfaction against the agreed specification	Acceptance was performed by the requirement originator

K.3 Target-User Evaluation Procedure

Table K.3: Target-user evaluation procedure used as the common session structure.

Step	Procedure
1	Obtain informed consent and assign the participant study code.
2	Select two of the eight available modules.
3	Work through the selected module content and interactive examples.
4	Use the visualiser, Code Space, practice, or other available features as required by the selected modules.
5	Complete the examination stage for each selected module.
6	Complete the applied knowledge check and relevant feature or agreement ratings.
7	Complete retrospective confidence ratings for perceived confidence before and after the session.
8	Complete open reflection and interface feedback.
9	Complete the System Usability Scale.
10	Collate responses under the anonymised study code for analysis.

K.4 Cycle 3 Instrument Revision

Table K.4: Changes between the Cycle 1 and Cycle 2 booklet and the Cycle 3 booklet.

Instrument area	Cycles 1 and 2	Cycle 3
Confidence	Indexing, slicing, immutability, comparison	Indexing, slicing, looping, comparison, tracing code step-by-step
Applied knowledge	Four questions covering indexing, slicing, immutability, comparison	Five questions with an additional method-mutation question
Agreement	Shorter general agreement set	Feature-level ratings for visualiser, highlighting, feedback, practice, localisation, Code Space, Challenges, and leaderboard
SUS	Ten-item SUS	Unchanged ten-item SUS

K.5 Aim-to-Evidence Traceability

Table K.5: Traceability from project aims to evidence and interpretation boundaries.

Aim	Primary evidence	Supporting evidence	Interpretation boundary
Aim 1	Eight modules and applied knowledge results	Requirements, development tests, target-user observations	Evidence covers implemented modules and tested content, not all possible learner outcomes
Aim 2	Visualiser and Code Space observations	Interpreter reference tests and peer feedback	Evidence supports implementation and usability, not long-term learning effectiveness
Aim 3	Applied knowledge check and misconception feedback	Practice Engine implementation, peer evaluation, UAT	Knowledge results are short-term and cohort dependent
Aim 4	Localisation ratings and implementation tests	Field-level fallback, transpiler tests, UAT	Independent linguistic quality review remains outstanding
Aim 5	Challenges and leaderboard reports and UAT	Lecturer requirement checks	Single-session use does not establish sustained engagement

K.6 Iterative Evaluation Traceability

Table K.6: Traceability from evaluation findings to design responses and later evidence.

Cycle	Finding	Design response	Later evidence
1	Need for audio support	Added spoken narration and completion-based progression	Narration became part of the implemented visualiser and remained available in later evaluation
1	Practice pool limited repetition and diagnosis	Replaced fixed practice with the Practice Engine	Practice became procedural across modules
1	Visualiser required a manual trigger	Added automatic playback on entering Visualise	Visualiser remained highly rated in later feedback
1	Failed visualiser evaluation could appear valid	Changed failure behaviour to report an error	Correctness boundary was retained in later implementation
2	Slicing explanation was ambiguous	Identified need for direct start, stop, and step explanation	Cycle 3 slicing performance remained a refinement target
2	Practice could feel repetitive	Identified need for broader practice coverage	Cycle 3 practice relevance was the lowest-rated core learning feature
3	isiXhosa terminology remained unclear for some participants	Identified independent language review as future work	Localisation remained an open refinement area
3	Some participants did not trust grading	Identified grading transparency as future refinement	Further review of examination and grading presentation is required

K.7 Limitations and Implications

Table K.7: Evaluation limitations and implications for future work.

Limitation	Effect on interpretation	Future response
Small samples	Descriptive results with limited generalisability	Use larger participant groups
Different academic levels	Limits direct cohort comparison	Use more closely matched participant groups
Cycle 3 followed a Java practical	Instructional context differs from earlier cycles	Use a consistent testing context
Revised Cycle 3 instrument	Some measures are not directly comparable	Retain a common core instrument across cycles
Retrospective confidence	Confidence is not a direct pre-post learning measure	Use matched pre- and post-session measures
Two of eight modules per participant	Evaluation is not exhaustive across the system	Rotate modules or test the full module set
Researcher present	Possible observer effect	Compare with less moderated or unmoderated sessions
Single-session evaluation	Sustained engagement cannot be established	Conduct longitudinal follow-up
Unscored expert review	No quantitative expert-usability measure	Add a structured expert instrument
Fixed examination-bank defects	Weakens evidence for the examination component	Revise and retest the bank
No independent isiXhosa review	Linguistic quality is not independently confirmed	Obtain independent language review
Browser speech limitations	isiXhosa narration remains unavailable	Evaluate alternative speech solutions
Incomplete rendering test coverage	Logic correctness does not guarantee visual correctness	Add component and end-to-end rendering tests

K.8 Evaluation Decision Log

Table K.8: Key evaluation decisions and their rationale.

Decision	Reason	Evidence	Consequence
Use iterative target-user evaluation	Learner feedback was expected to reveal interaction requirements not fully captured during specification	Cycle 1 generated new interface requirements	The artefact changed between evaluation cycles
Retain SUS unchanged	Preserve direct usability comparison across cycles	SUS is a standardised ten-item instrument	SUS remained directly comparable while other booklet sections changed
Report knowledge separately from confidence	The measures capture different constructs and showed divergence	Cycle 3 confidence increased while some knowledge results decreased relative to Cycle 2	Claims about learning were kept cautious
Treat Cycle 3 separately	Participant population, instructional context, and instrument changed	Evaluation-cycle conditions reported above	Cycle 3 is not treated as a controlled replication
Continue the misconception catalogue	Recurring errors appeared across independently recruited cohorts	Applied knowledge and qualitative responses	Catalogue remains a living design input
Retain qualitative peer feedback	Open responses exposed defects not captured by target-user sessions	Three PyTutor developer forms	Examination-bank refinement became a priority

L Interface Screenshots

The following screenshots document the principal learner-facing areas of String Conundrum in the implemented system. They are included as supplementary evidence of the completed interface and do not form part of the ten-page main-paper body.

1. Learning Modules

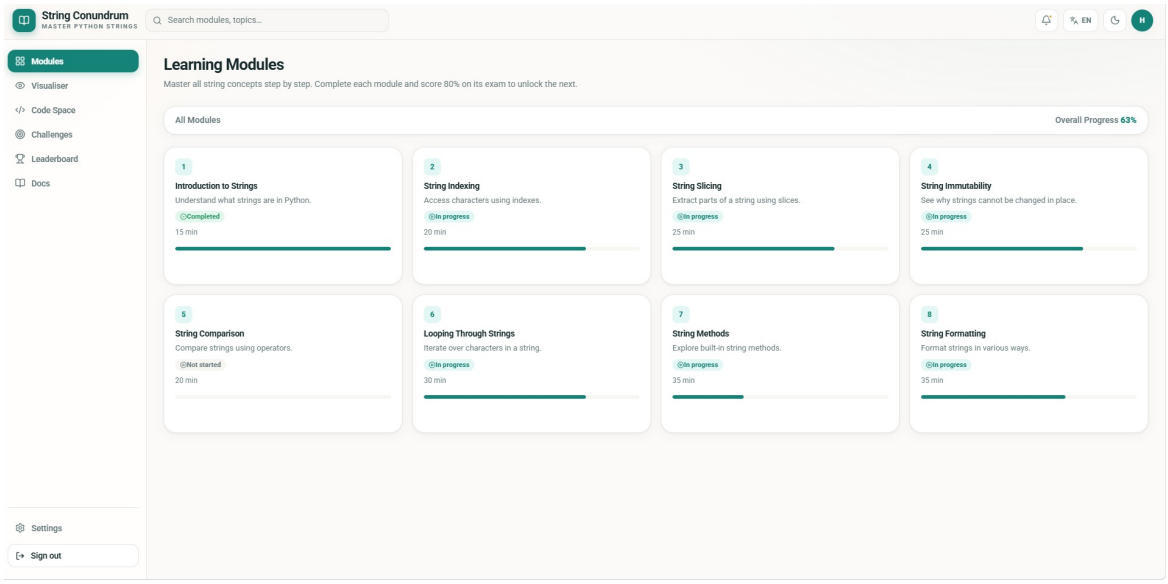


Figure L.1: Learning Modules overview showing the eight string-topic modules, module progress, estimated completion times, and overall learner progress.

2. String Visualiser

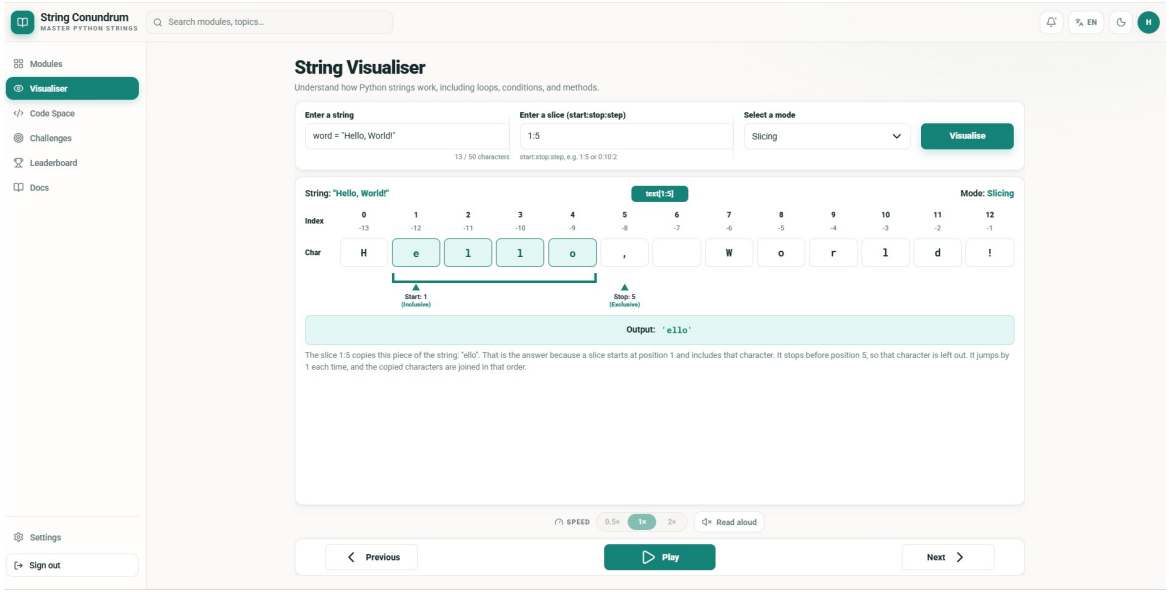


Figure L.2: Standalone String Visualiser showing simultaneous positive and negative indices, slice boundaries, the selected substring, output, and playback controls.

3. Code Space

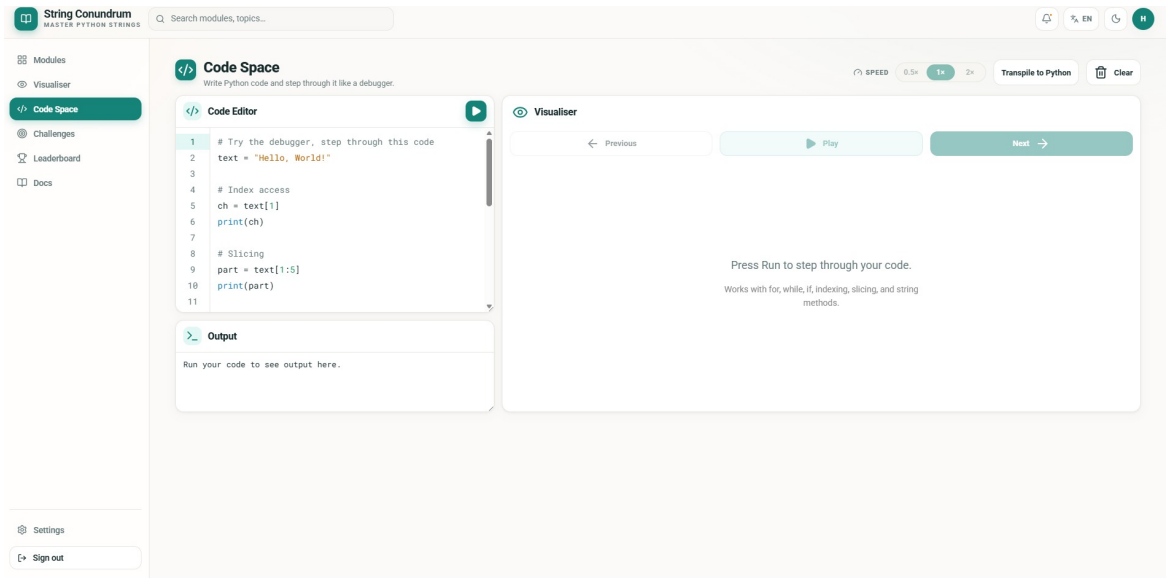


Figure L.3: Code Space with the Python editor, execution output area, and integrated visualiser pane.

4. Challenges

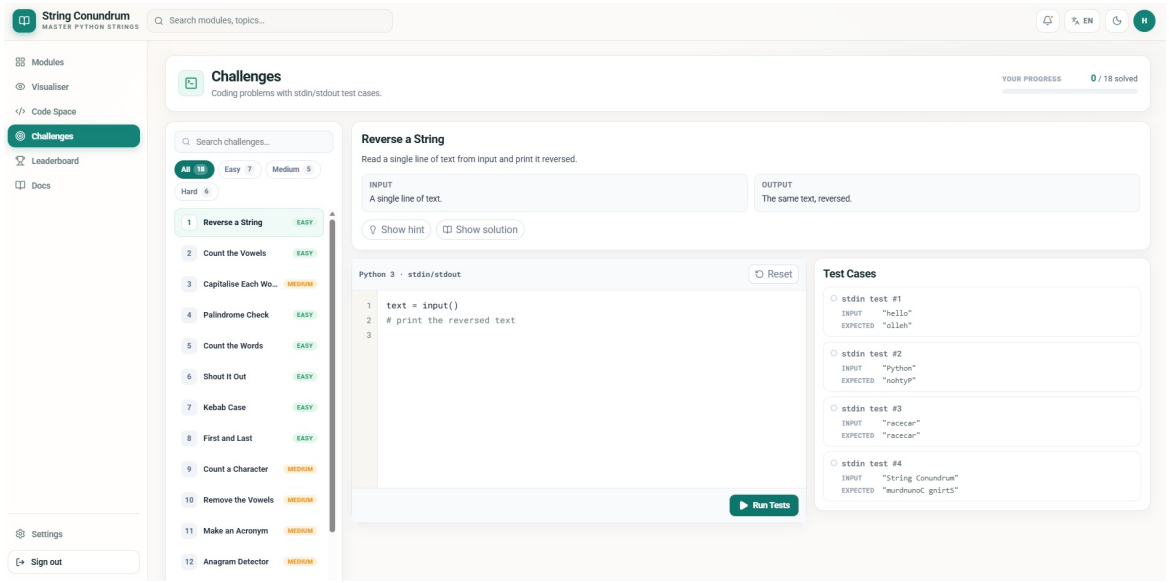


Figure L.4: Challenges interface showing difficulty-filtered problems, the code editor, hints and solutions, test cases, and the run-tests action.

5. Leaderboard

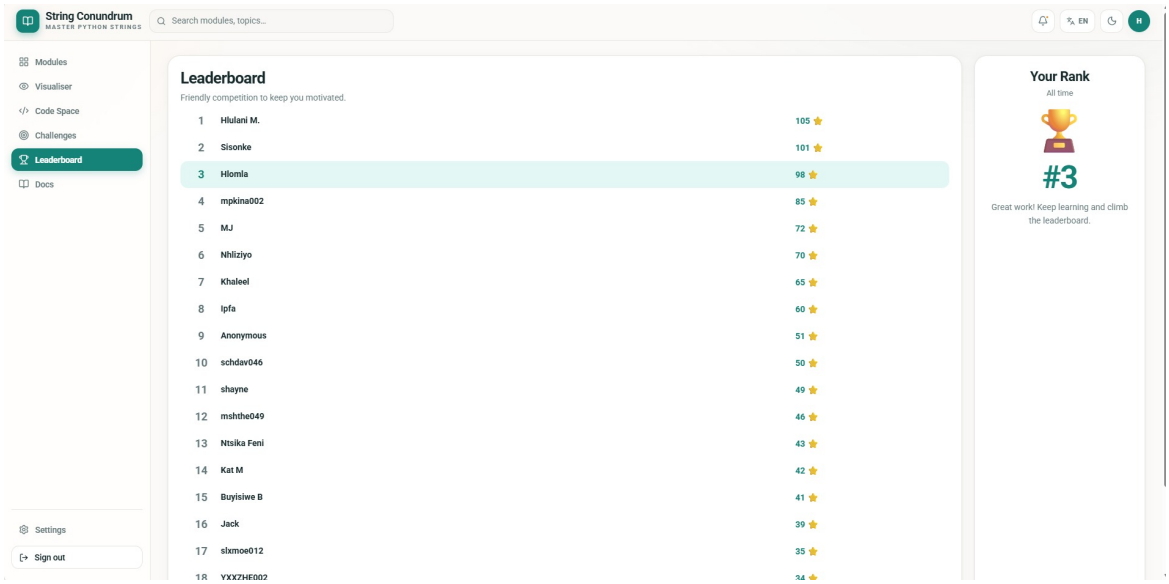


Figure L.5: Leaderboard interface showing ranked participation and the signed-in learner's current position.

6. isiXhosa ↔ Python Reference (Docs)

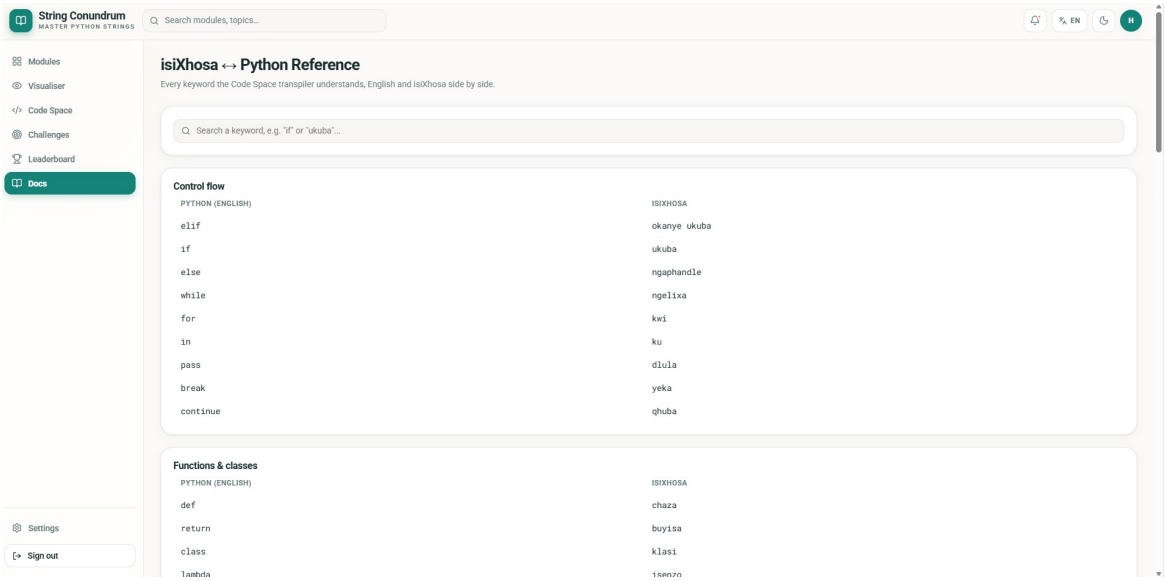


Figure L.6: isiXhosa–Python reference showing corresponding keywords used by the deterministic transpiler.

7. Introduction to Strings (Module 1)

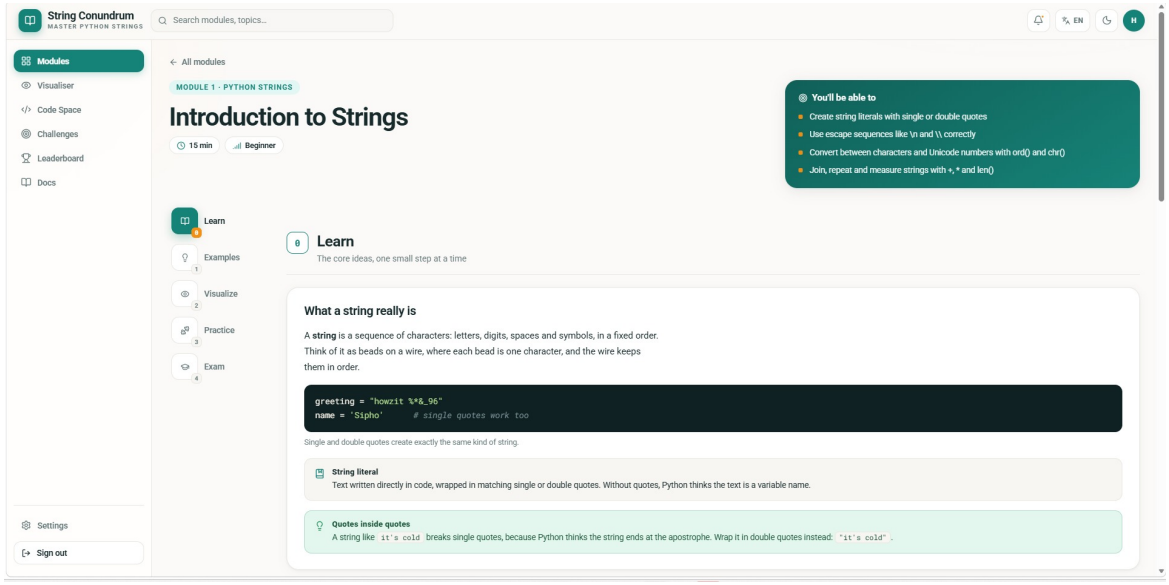


Figure L.7: Introduction to Strings module showing the staged learning interface, learning outcomes, explanatory content, and worked code example.

8. Settings

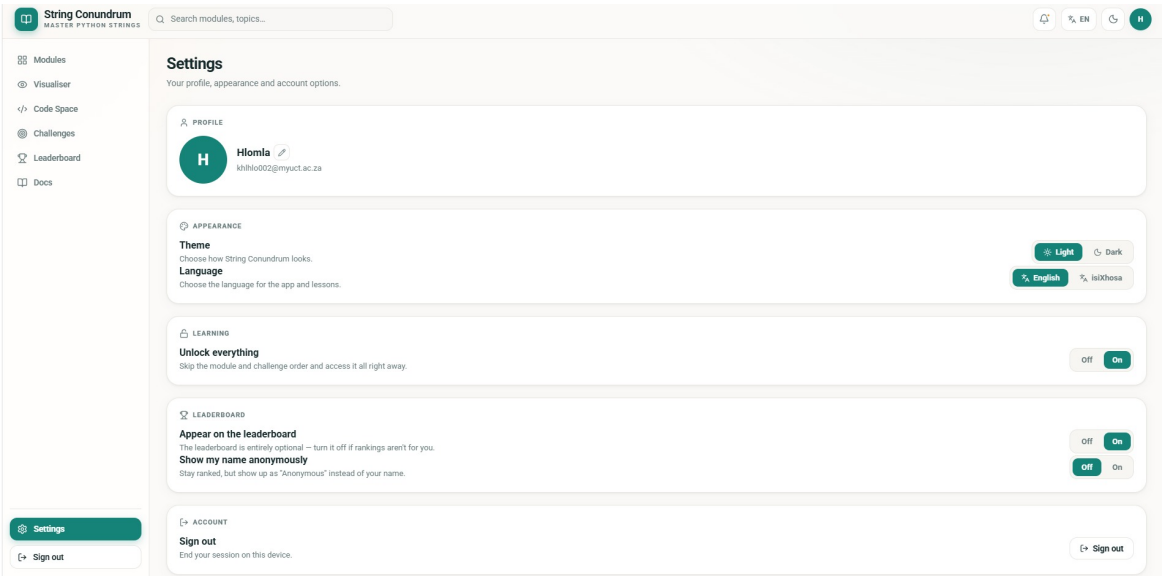


Figure L.8: Settings interface showing profile information, theme and language controls, module-unlock preference, and learner-controlled leaderboard privacy settings.